Codebook

Contents

1 basic **2**
- 1.1 build . . . 2
- 1.2 default . . . 2
- 1.3 division . . . 2
- 1.4 fast . . . 2
- 1.5 pragma . . . 2
- 1.6 random . . . 2

2 data structure **2**
- 2.1 2D Segment Tree . . . 2
- 2.2 Binary Indexed Tree . . . 3
- 2.3 bitset . . . 3
- 2.4 Li Chao Segment Tree . . . 3
- 2.5 pbds . . . 3
- 2.6 Range Set . . . 3
- 2.7 Sparse Table . . . 3
- 2.8 Standard Template Library . . . 4
- 2.9 Treap . . . 4
- 2.10 Undo Disjoint Set . . . 4
- 2.11 Unorder Map . . . 4

3 flow matching **4**
- 3.1 Dinic . . . 4
- 3.2 flow building . . . 5
- 3.3 matching . . . 6
- 3.4 matching general . . . 6
- 3.5 matching general weight . . . 6
- 3.6 Minimum Cost Maximum Flow . . . 7

4 geometry **8**
- 4.1 basic . . . 8
- 4.2 convex hull . . . 8
- 4.3 convex hull dist . . . 8
- 4.4 Minimum Enclosing Circle . . . 8
- 4.5 Minkowski . . . 8
- 4.6 Point In Convex . . . 8
- 4.7 Point In Poly . . . 9
- 4.8 Point In Triangle . . . 9
- 4.9 Point Segment Dist . . . 9

5 graph **9**
- 5.1 2SAT . . . 9
- 5.2 bridge . . . 10
- 5.3 bridge bcc . . . 10
- 5.4 cut . . . 10
- 5.5 cut bcc . . . 10
- 5.6 prim . . . 10
- 5.7 spfa . . . 10
- 5.8 Strongly Connected Component . . . 11

6 math **11**
- 6.1 big number . . . 11
- 6.2 burnside lemma . . . 11
- 6.3 catalan . . . 11
- 6.4 Chinese Remainder Theorem coprime . . . 11
- 6.5 Chinese Remainder Theorem coprime . . . 12
- 6.6 Chinese Remainder Theorem not coprime . . . 12
- 6.7 count prime . . . 12
- 6.8 det . . . 12
- 6.9 extgcd . . . 12
- 6.10 factorize . . . 12
- 6.11 fraction . . . 13
- 6.12 gaussian elimination . . . 13
- 6.13 josephus . . . 13
- 6.14 lucas . . . 13
- 6.15 math block . . . 13
- 6.16 miller rabin . . . 14
- 6.17 phi . . . 14
- 6.18 phi lemma . . . 14
- 6.19 primes er . . . 14
- 6.20 primes linear . . . 14
- 6.21 two equations . . . 14

7 other **14**
- 7.1 aliens dp . . . 14
- 7.2 all LCS . . . 15
- 7.3 dc dp . . . 15
- 7.4 digit dp . . . 15
- 7.5 geometry . . . 15
- 7.6 line container . . . 15
- 7.7 mo . . . 16
- 7.8 number split . . . 16
- 7.9 python . . . 16
- 7.10 sos dp . . . 16
- 7.11 subset . . . 16
- 7.12 xor basis . . . 16

8 polynomial **16**
- 8.1 FFT . . . 16
- 8.2 NTT . . . 16
- 8.3 NTT prime . . . 17
- 8.4 operation . . . 17

9 string **18**
- 9.1 hash . . . 18
- 9.2 kmp . . . 18
- 9.3 manacher . . . 18
- 9.4 min rotation . . . 19
- 9.5 string trie . . . 19
- 9.6 suffix array . . . 19
- 9.7 xor trie . . . 19
- 9.8 z . . . 20

10 tree **20**
- 10.1 Heavy Light Decomposition . . . 20

1 basic

1.1 build

```
code tem.cpp
...
for i in {A..M}; do cp tem.cpp \$i.cpp; done;
code *.cpp
code run

寫入以下編譯與執行指令至 run 檔案中：
g++ -std=c++20 -DABS -Wall -Wextra -Wshadow $1.cpp -o $1 && ./
    $1

chmod +x run
```

1.2 default

```
#include <bits/stdc++.h>
#define int long long
#define pii pair<int, int>
#define F first
#define S second
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define FOR(i, a, b) for(int i = a; i <= b; ++i)
#define IO ios::sync_with_stdio(0), cin.tie(0)
using namespace std;
void debug() { cerr << endl; }
template <typename T, typename ...U>
void debug(T i, U ...j) { cerr << i << ' ', debug(j...); }
#define test(...) debug("[" #__VA_ARGS__ "]:", __VA_ARGS__)

signed main() {
  IO;

}
```

1.3 division

```
ll floor(ll a, ll b) {
  return a / b - (a < 0 && a % b);
}
ll ceil(ll a, ll b) {
  return a / b + (a > 0 && a % b);
}
a / b <  x -> floor(a, b) + 1 <= x
a / b <= x -> ceil(a, b) <= x
x <  a / b -> x <= ceil(a, b) - 1
x <= a / b -> x <= floor(a, b)
```

1.4 fast

```
#pragma GCC optimize("Ofast,inline,unroll-loops")
#pragma GCC target("bmi,bmi2,lzcnt,popcnt,avx2")
#define windows_system
#ifdef windows_system
/* For Windows */
inline int getchar_unlocked() { return getchar(); }
inline void putchar_unlocked(char _c) { putchar(_c); }
#endif
inline int in() {
  int re = 0;
  char c = getchar_unlocked();
  while (c == ' ' || c == '\n') c = getchar_unlocked();
  while (c >= '0' && c <= '9') re = (re << 3) + (re << 1) + c -
       '0', c = getchar_unlocked();
  return re;
}
inline void out(int x) {
  char str[20];
  int pos = 0;
  do {
    str[pos++] = x % 10 + '0';
    x /= 10;
  } while (x);
  for (int i = pos - 1; i >= 0; i--) putchar_unlocked(str[i]);
  putchar_unlocked('\n');
}
```

1.5 pragma

```
#pragma GCC optimize("Ofast,no-stack-protector")
#pragma GCC optimize("no-math-errno,unroll-loops")
#pragma GCC target("sse,sse2,sse3,ssse3,sse4")
#pragma GCC target("popcnt,abm,mmx,avx,arch=skylake")
```

1.6 random

```
mt19937 seed(chrono::steady_clock::now().time_since_epoch().
    count());
inline int rnd(int l, int r) {
  return uniform_int_distribution<int>(l, r)(seed);
}
inline double drnd(double l, double r) {
  return uniform_real_distribution<double>(l, r)(seed);
}
```

2 data structure

2.1 2D Segment Tree

```
struct Node2 {
  int val; Node2 *lc, *rc;
  Node2 () {
    val = 0;
    lc = rc = nullptr;
  }
};
struct Node1 {
  Node2 *c; Node1 *lc, *rc;
  Node1 () {
    c = nullptr;
    lc = rc = nullptr;
  }
};
int N;
int Val2(Node2 *n2) { return (n2 ? n2->val : 0ll); }
void pull2(Node2 *n2) { n2->val = Val2(n2->lc) + Val2(n2->rc);
    }
void update2(Node2 *n2, int l, int r, int id, int v) {
  if (l == r) {
    n2->val += v; return;
  }
  int mid = (l + r) >> 1;
  if (id <= mid) {
    if (!n2->lc) n2->lc = new Node2();
    update2(n2->lc, l, mid, id, v);
  } else {
    if (!n2->rc) n2->rc = new Node2();
    update2(n2->rc, mid + 1, r, id, v);
  }
  pull2(n2);
}
void update1(Node1 *n1, int l, int r, int qx, int qy, int v) {
  if (!n1->c) n1->c = new Node2();
  update2(n1->c, 1, N, qy, v);
  if (l == r) return;
  int mid = (l + r) >> 1;
  if (qx <= mid) {
    if (!n1->lc) n1->lc = new Node1();
    update1(n1->lc, l, mid, qx, qy, v);
  } else {
    if (!n1->rc) n1->rc = new Node1();
    update1(n1->rc, mid + 1, r, qx, qy, v);
  }
}
int query2(Node2 *n2, int l, int r, int ql, int qr) {
  if (!n2) return 0;
  if (ql <= l && r <= qr) return n2->val;
  int mid = (l + r) >> 1;
  if (qr <= mid)
    return query2(n2->lc, l, mid, ql, qr);
  else if (mid + 1 <= ql)
    return query2(n2->rc, mid + 1, r, ql, qr);
  else
    return query2(n2->lc, l, mid, ql, qr) + query2(n2->rc, mid
        + 1, r, ql, qr);
}
int query1(Node1 *n1, int l, int r, int qxl, int qxr, int qyl,
    int qyr) {
  if (!n1) return 0;
  if (qxl <= l && r <= qxr) return query2(n1->c, 1, N, qyl, qyr
      );
  int mid = (l + r) >> 1;
  if (qxr <= mid)
    return query1(n1->lc, l, mid, qxl, qxr, qyl, qyr);
  else if (mid + 1 <= qxl)
```

```
    return query1(n1->rc, mid + 1, r, qxl, qxr, qyl, qyr);
  else
    return query1(n1->lc, l, mid, qxl, qxr, qyl, qyr) + query1(
        n1->rc, mid + 1, r, qxl, qxr, qyl, qyr);
}
```

2.2 Binary Indexed Tree

```
struct BIT {
  int N;
  vector<int> info;
  BIT(int n) : N(n), info(n + 1) {}
  void update(int idx, int val) {
    for (int i = idx; i <= N; i += i & -i)
      info[i] += val;
  }
  int query(int idx) {
    int res = 0;
    for (int i = idx; i > 0; i -= i & -i)
      res += info[i];
    return res;
  }
  int kth(int k) { // kth small
    int res = 0;
    for (int i = 1 << __lg(N); i > 0; i >>= 1) {
      if (res + i <= N && info[res + i] < k) {
        res += i;
        k -= info[res];
      }
    }
    return res + 1;
  }
};
```

2.3 bitset

```
#pragma GCC target("popcnt")
// 15 = 1111
bitset <10> bit(15);                // 宣告長度 = 10 的 bitset
cout << bit.count() << "\n";        // 回傳幾個 1
cout << bit.none() << "\n";         // 回傳是否全為 0
cout << bit.any() << "\n";          // 回傳是否有 1
cout << bit._Find_first() << "\n"; // 回傳第一個 bit = 1 的位置
int x = 2;
cout << bit._Find_next(x) << "\n"; // 回傳 x 後，第一個 bit = 1 的位置
```

2.4 Li Chao Segment Tree

```
// MIN LeeChao Segment Tree
struct line {
  int m, k;
  line *lc, *rc;
  line() { m = 0, k = INT_MAX, lc = nullptr, rc = nullptr; }
  line(int _m, int _k) { m = _m, k = _k, lc = nullptr, rc =
      nullptr; }
  int calc(int x) {
    return m * x + k;
  }
  void push(int l, int r) {
    if (l == r) return;
    if (!lc) lc = new line();
    if (!rc) rc = new line();
  }
};
struct Lee_Chao {
  line* root;
  Lee_Chao() : root(new line()) {}
  void update(line* node, int l, int r, line add) {
    if (l == r) {
      if (add.calc(l) < node->calc(l)) {
        swap(add.m, node->m);
        swap(add.k, node->k);
      }
      return;
    }
    // 預設 seg[node] 的斜率 < add 的斜率
    if (node->m > add.m) {
      swap(add.m, node->m);
      swap(add.k, node->k);
    }
    node->push(l, r);
    int mid = (l + r) >> 1;
    // 將小的 f(x) 留下
    if (node->calc(mid) > add.calc(mid)) {
      swap(add.m, node->m);
      swap(add.k, node->k);
      update(node->rc, mid + 1, r, add);
    } else {
      update(node->lc, l, mid, add);
    }
  }
  int query(line* node, int l, int r, int x) {
    if (node->k == INT_MAX) return INT_MAX;
    if (l == r) return node->calc(x);
    int mid = (l + r) >> 1;
    if (x <= mid)
      return min(node->calc(x), query(node->lc, l, mid, x));
    else
      return min(node->calc(x), query(node->rc, mid + 1, r, x))
          ;
  }
  void update(line l) { update(root, 1, N, l); }
  int query(int x) { return query(root, 1, N, x); }
};
```

2.5 pbds

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
void solve() {
  tree<int, null_type, greater<int>, rb_tree_tag,
       tree_order_statistics_node_update> bst;
  bst.insert(1);
  int a = *bst.find_by_order(2); // 第 k + 1 大的數
  int b = bst.order_of_key(2);   // 有幾個數 > k
  // -----------------------------------------------
  tree<int, null_type, less<int>, rb_tree_tag,
       tree_order_statistics_node_update> bst;
  bst.insert(1);
  int a = *bst.find_by_order(2); // 第 k + 1 小的數
  int b = bst.order_of_key(2);   // 有幾個數 < k
}
```

2.6 Range Set

```
struct RangeSet { // [l, r)
  set<pii> st;
  void cut(int x) {
    auto it = st.lower_bound({x + 1, -1});
    if (it == st.begin()) return;
    auto [l, r] = *prev(it);
    if (l >= x || x >= r) return;
    st.erase(prev(it));
    st.insert({l, x});
    st.insert({x, r});
  }
  vector<pii> split(int l, int r) {
    // remove and return ranges in [l, r)
    cut(l), cut(r);
    vector<pii> res;
    while (true) {
      auto it = st.lower_bound({l, -1});
      if (it == st.end() || r <= it->first) break;
      res.push_back(*it), st.erase(it);
    }
    return res;
  }
  void insert(int l, int r) {
    // add a range [l, r), [l, r) not in st
    auto it = st.lower_bound({l, r});
    if (it != st.begin() && prev(it)->second == l)
      l = prev(it)->first, st.erase(prev(it));
    if (it != st.end() && r == it->first)
      r = it->second, st.erase(it);
    st.insert({l, r});
  }
  bool count(int x) {
    auto it = st.lower_bound({x + 1, -1});
    return it != st.begin() && prev(it)->first <= x && x < prev
        (it)->second;
  }
};
```

2.7 Sparse Table

```
int a[N];
struct Sparse_table {
  int n;
  vector<vector<int>> st;
```

```
  Sparse_table(int _n) : n(_n), st(_n + 1, vector<int>(__lg(_n)
      + 1)) {
    for (int i = 1; i <= n; i++) st[i][0] = a[i];
    for (int lg = 1; lg <= __lg(n); lg++) {
      int len = 1 << (lg - 1);
      for (int i = 1; i + len <= n; i++)
        st[i][lg] = max(st[i][lg - 1], st[i + len][lg - 1]);
    }
  }
  int query(int l, int r) {
    int lg = __lg(r - l + 1);
    int len = 1 << lg;
    return max(st[l][lg], st[r - len + 1][lg]);
  }
};
```

2.8 Standard Template Library

```
struct Data {
  int x, y;
};
// priority_queue
struct cmp {
  bool operator()(Data a, Data b) {
    return a.x < b.x;
  }
};
priority_queue<Data, vector<Data>, cmp> pq;
// set
struct cmp {
  bool operator() (Data p, Data q) {
    return p.x < q.x;
  }
};
set<Data, cmp> st;
```

2.9 Treap

```
struct node {
  int data, sz;
  node *l, *r;
  node(int k) : data(k), sz(1), l(0), r(0) {}
  void up() {
    sz = 1;
    if (l) sz += l->sz;
    if (r) sz += r->sz;
  }
  void down() {}
};
int sz(node *a) { return a ? a->sz : 0; }
node *merge(node *a, node *b) {
  if (!a || !b) return a ? a : b;
  if (rand() % (sz(a) + sz(b)) < sz(a))
    return a->down(), a->r = merge(a->r, b), a->up(), a;
  return b->down(), b->l = merge(a, b->l), b->up(), b;
}
void split1(node *o, node *&a, node *&b, int k) {
  if (!o) return a = b = 0, void();
  o->down();
  if (o->data <= k)
    a = o, split1(o->r, a->r, b, k), a->up();
  else
    b = o, split1(o->l, a, b->l, k), b->up();
}
void split2(node *o, node *&a, node *&b, int k) {
  if (sz(o) <= k) return a = o, b = 0, void();
  o->down();
  if (sz(o->l) + 1 <= k)
    a = o, split2(o->r, a->r, b, k - sz(o->l) - 1);
  else
    b = o, split2(o->l, a, b->l, k);
  o->up();
}
node *kth(node *o, int k) {
  if (k <= sz(o->l)) return kth(o->l, k);
  if (k == sz(o->l) + 1) return o;
  return kth(o->r, k - sz(o->l) - 1);
}
int Rank(node *o, int key) {
  if (!o) return 0;
  if (o->data < key)
    return sz(o->l) + 1 + Rank(o->r, key);
  else
    return Rank(o->l, key);
}
bool erase(node *&o, int k) {
  if (!o) return 0;
  if (o->data == k) {
    node *t = o;
    o->down(), o = merge(o->l, o->r);
    delete t;
    return 1;
  }
  node *&t = k < o->data ? o->l : o->r;
  return erase(t, k) ? o->up(), 1 : 0;
}
void insert(node *&o, int k) {
  node *a, *b;
  split(o, a, b, k),
    o = merge(a, merge(new node(k), b));
}
void interval(node *&o, int l, int r) {
  node *a, *b, *c;
  split2(o, a, b, l - 1), split2(b, b, c, r);
  // operate
  o = merge(a, merge(b, c));
}
```

2.10 Undo Disjoint Set

```
struct DSU {
  vector<int> parent, size, tag;
  vector<tuple<int, int, int, int, int>> history;
  DSU(int n) {
        parent.resize(n + 1);
        size.resize(n + 1, 1);
        tag.resize(n + 1, 0);
        iota(all(parent), 0);
  }
  int find(int x) {
        if (parent[x] == x) return x;
        return find(parent[x]);
  }
  bool merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (size[x] < size[y]) swap(x, y);
        history.pb({x, y, parent[y], tag[x], size[x]});
        parent[y] = x;
        size[x] += size[y];
        return true;
  }
  void add_tag(int head, int val) {
        tag[head] += val;
  }
  void undo() {
        auto [px, py, parent_py, base, sz_x] = history.back();
        parent[py] = parent_py;
        size[px] = sz_x;
        int delta = tag[px] - base;
        tag[py] += delta;
        history.pop_back();
  }
};
```

2.11 Unorder Map

```
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t a) const {
    static const uint64_t FIXED_RANDOM = chrono::steady_clock::
        now().time_since_epoch().count();
    return splitmix64(i + FIXED_RANDOM);
  }
};
unordered_map <int, int, custom_hash> m1;
```

3 flow matching

3.1 Dinic

```
struct Flow {
  struct Edge {
    // rc : residual capcity
    int u, rc, rev;
  };
  vector<vector<Edge>> G;
  vector<int> dis, it;
  Flow(int n) : G(n), dis(n), it(n) {}
  void add(int v, int u, int c) {
    G[v].push_back({u, c, sz(G[u])});
    G[u].push_back({v, 0, sz(G[v]) - 1});
  }
```

```
  int dfs(int v, int t, int f) {
    if (v == t || f == 0) return f;
    for (int &i = it[v]; i < sz(G[v]); i++) {
      auto &[u, rc, rev] = G[v][i];
      if (dis[u] != dis[v] + 1) continue;
      int res = dfs(u, t, min(f, rc));
      if (res <= 0) continue;
      rc -= res;
      G[u][rev].rc += res;
      return res;
    }
    return 0;
  }
  int flow(int s, int t) {
    int ans = 0;
    for (int l = 30; l >= 0; l--) while (true) {
      fill(all(dis), INT_MAX);
      queue<int> Q;
      Q.push(s);
      dis[s] = 0;
      while (! Q.empty()) {
        int v = Q.front(); Q.pop();
        for (auto [u, rc, rev] : G[v]) {
          if ((rc >> l) <= 0 || dis[u] < INT_MAX) continue;
          dis[u] = dis[v] + 1;
          Q.push(u);
        }
      }
      if (dis[t] == INT_MAX) break;

      fill(all(it), 0);
      while (true) {
        int res = dfs(s, t, INT_MAX);
        if (res <= 0) break;
        ans += res;
      }
    }
    return ans;
  }
};
```

3.2 flow building

Dinic

基本建邊

- 有向邊 $u \to v$，容量 c：`add(u, v, c)`，反向邊容量為 0。
- 無向邊 $u - v$，容量 c：`add(u, v, c)`，並把反向邊容量也設為 c，不需另外再加一條邊。
- 多源點或多匯點：開超級源 S，連 $S \to$ 每個源點 (∞)；開超級匯 T，連每個匯點 $\to T$ (∞)。
- 點有容量 c：拆點為 $v_{in} \to v_{out}$ (c)，進入的邊接到 v_{in}，出去的邊從 v_{out} 出發。

匹配與分配

- 二分圖匹配：$S \to L$ (1)、$L \to R$ (1)、$R \to T$ (1)，最大流即最大匹配數。
- 每個點最多配 k 個：把 S 側或 T 側的邊容量改為 k。
- 同一對物品只能配一次：中間的邊容量設為 1。
- 多對多分配：$S \to$ 人 (可做件數)、人 $\to$ 工作 (1 或上限)、工作 $\to T$ (需求量)；是否可行看是否滿流。

最小割模型

- 題目出現「代價」「二選一」「放棄」等字眼時，考慮最小割。
- 某條邊不能被割：容量設為 ∞。
- 最大權閉合子圖 (選 i 就必須選 j)：
  - 收益 $p > 0$：$S \to i$ (p)
  - 花費 c：$i \to T$ (c)
  - 依賴關係：$i \to j$ (∞)
  - 答案 = 正收益總和 − 最大流
- 二選一 (每點選 A 或 B，相鄰兩點選得不同要付罰金 w)：
  - 規定點在 S 側代表選 A，在 T 側代表選 B。
  - $S \to i$ (選 B 的代價)、$i \to T$ (選 A 的代價)
  - 兩點選得不同的罰金：$i \to j$ (w) 與 $j \to i$ (w)
  - 答案 = 最小割
- 強制某點一定選 A 或一定選 B：$S \to i$ (∞) 或 $i \to T$ (∞)。
- 網格或棋盤類：先黑白染色成二分圖。
  - 最小點覆蓋 = 最大匹配
  - 最大獨立集 = 總點數 − 最大匹配

路徑類

- 邊不相交的路徑數：每條邊容量 1，直接跑最大流。
- 點不相交的路徑數：拆點，$v_{in} \to v_{out}$ (1)。
- DAG 最小路徑覆蓋：
  - 每點拆成出點與入點：$S \to u_{out}$ (1)、$u_{in} \to T$ (1)
  - 原圖的邊 $u \to v$：$u_{out} \to v_{in}$ (1)
  - 答案 = n − 最大流
  - 若路徑可重複經過點，先做遞移閉包再建圖。
- 與時間有關 (每單位時間走一步、各時刻容量不同)：建分層圖，把每個點依時間複製成 (v, t)，邊從第 t 層連到第 $t+1$ 層。

有上下界的流 (邊 $u \to v$ 的流量需在 $[l, r]$ 內)

- 建 $u \to v$ ($r - l$)，並記錄 $in[v] \mathrel{+}= l$、$out[u] \mathrel{+}= l$。
- 另開源匯 S'、T'：
  - 若 $in[x] - out[x] > 0$：$S' \to x$ (差值)
  - 若差值 < 0：$x \to T'$ (− 差值)
- 若題目本身有源匯 s, t，再加 $t \to s$ (∞)。
- 從 S' 到 T' 跑滿流才代表有可行解。
- 求最大流：接著在殘量網路上跑 $s \to t$。
- 求最小流：接著在殘量網路上跑 $t \to s$，再用原流量減去。

MCMF

基本建邊

- 每條邊為 `add(u, v, cap, cost)`，反向邊容量 0、費用 $-cost$。

常見情境

- 帶權二分匹配 (指派問題)：$S \to L$ (1, 0)、$L \to R$ (1, w)、$R \to T$ (1, 0)。
- 求最大收益：費用取負 $cost = -w$，最後答案再取負。
- 只求收益最大、不要求流量最大：
  - 做法一：增廣時一旦最短路費用 ≥ 0 就停止。
  - 做法二：每個點加一條 $\to T$ (1, 0) 的邊，讓它可以選擇不配。
- 指定總流量為 F：開新源 S_0，連 $S_0 \to S$ (F, 0)。
- 點有費用或收益：拆點 $v_{in} \to v_{out}$ (容量, 費用)；例如每點只能拿一次收益 p，建 (1, $-p$)。
- 點可經過多次，但收益只能拿一次：拆點並放兩條平行邊 (1, $-p$) 與 (∞, 0)。
- 邊際成本遞增 (例如費用為 x^2)：拆成多條容量 1 的平行邊，費用依序為 $1, 3, 5, \ldots$ (即 x^2 的差分)。前提是費用為凸，一條比一條貴，才會依序被選用。
- k 條路徑的最大權 (每格只能取一次，共走 k 次)：格子拆點並放兩條平行邊 (1, $-val$) 與 (k − 1, 0)，再連 $S \to$ 起點 (k, 0)。
- 區間選擇 (選帶權區間，每個位置最多被 k 個區間覆蓋)：
  - 座標離散化後建一條鏈：$i \to i+1$ (k, 0)。
  - 每個區間 $[l, r]$：$l \to r$ (1, $-w$)。
  - 從鏈頭到鏈尾跑流量 k。
- 資源用完後隔一段時間可再用 (餐巾問題類型)：
  - 按天拆成「用完」與「需要」兩類點。
  - $S \to$ 第 i 天用完 (用量, 0)、第 i 天需要 $\to T$ (需求, 0)。
  - 買新的：$S \to$ 需要 (∞, 買價)

- 快速回收：用完 $i \to$ 需要 $i+a$ (∞, 快速費用)
- 慢速回收：用完 $i \to$ 需要 $i+b$ (∞, 慢速費用)
- 留到隔天：用完 $i \to$ 用完 $i+1$ ($\infty, 0$)

- 某條邊一定要走：費用設為 $-\text{BIG}$，或改用上下界流。

注意事項

- Dinic's 算法：時間複雜度 $O(V^2E)$
- MCMF：時間複雜度 $O(FEV)$，其中 F 為最大流量。

建模方式對照表

題目關鍵字	建模方式
配對、分配、容量上限	最大流
點不能重複用、點有限制	拆點
最少代價阻止某事、二選一、依賴關係	最小割 (∞ 表示不能切)
配對或運送時帶權重、成本	MCMF
費用隨數量遞增	拆成平行邊，費用遞增
跟時間步有關	分層圖
至少、必須	上下界流

3.3 matching

```cpp
const int N = 2005;
vector <int> G[N];
struct Matching {
  int n;
  vector<bool> vis; vector<int> match;
  Matching(int _n) : n(_n), vis(_n + 1), match(_n + 1, -1) {}
  bool dfs(int v) {
    vis[v] = true;
    for (int u : G[v])
      if (match[u] == -1 || (! vis[match[u]] && dfs(match[u]))) {
        match[u] = v;
        return true;
      }
    return false;
  }
  int solve() {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
      vis.assign(n + 1, 0);
      if (dfs(i)) cnt++;
    }
    return cnt;
  }
};
```

3.4 matching general

```cpp
// O(V^3)
// if graph.match[i] == n, node i is not matched
// else, i is matched with graph.match[i]
struct Matching { // 0-based
  int fa[N], pre[N], match[N], s[N], v[N], n, tk;
  vector <int> g[N];
  queue <int> q;
  int Find(int u) {
    return u == fa[u] ? u : fa[u] = Find(fa[u]);
  }
  int lca(int x, int y) {
    tk++;
    x = Find(x), y = Find(y);
    for (; ; swap(x, y)) {
      if (x != n) {
        if (v[x] == tk) return x;
        v[x] = tk;
        x = Find(pre[match[x]]);
      }
    }
  }
  void blossom(int x, int y, int l) {
    while (Find(x) != l) {
      pre[x] = y, y = match[x];
      if (s[y] == 1) q.push(y), s[y] = 0;
      if (fa[x] == x) fa[x] = l;
      if (fa[y] == y) fa[y] = l;
      x = pre[y];
    }
  }
  bool bfs(int r) {
    for (int i = 0; i <= n; ++i) fa[i] = i, s[i] = -1;
    while (!q.empty()) q.pop();
    q.push(r);
    s[r] = 0;
    while (!q.empty()) {
      int x = q.front(); q.pop();
      for (int u : g[x]) {
        if (s[u] == -1) {
          pre[u] = x, s[u] = 1;
          if (match[u] == n) {
            for (int a = u, b = x, last; b != n; a = last, b = pre[a])
              last = match[b], match[b] = a, match[a] = b;
            return true;
          }
          q.push(match[u]);
          s[match[u]] = 0;
        } else if (!s[u] && Find(u) != Find(x)) {
          int l = lca(u, x);
          blossom(x, u, l);
          blossom(u, x, l);
        }
      }
    }
    return false;
  }
  int solve() {
    int res = 0;
    for (int x = 0; x < n; ++x) {
      if (match[x] == n) res += bfs(x);
    }
    return res;
  }
  void init(int _n) {
    n = _n, tk = 0;
    for (int i = 0; i <= n; ++i) match[i] = pre[i] = n;
    for (int i = 0; i < n; ++i) g[i].clear(), v[i] = 0;
  }
  void add_edge(int u, int v) {
    g[u].push_back(v), g[v].push_back(u);
  }
};
```

3.5 matching general weight

```cpp
// O(V^3)
// if graph.match[i] == 0, node i is not matched
// else, i is matched with graph.match[i]
struct WeightGraph { // 1-based
  static const int inf = INT_MAX;
  static const int maxn = 514;
  struct edge {
    int u, v, w;
    edge(){}
    edge(int u, int v, int w): u(u), v(v), w(w) {}
  };
  int n, n_x;
  edge g[maxn * 2][maxn * 2];
  int lab[maxn * 2];
  int match[maxn * 2], slack[maxn * 2], st[maxn * 2], pa[maxn * 2];
  int flo_from[maxn * 2][maxn + 1], S[maxn * 2], vis[maxn * 2];
  vector<int> flo[maxn * 2];
  queue<int> q;
  int e_delta(const edge &e) { return lab[e.u] + lab[e.v] - g[e.u][e.v].w * 2; }
  void update_slack(int u, int x) { if (!slack[x] || e_delta(g[u][x]) < e_delta(g[slack[x]][x])) slack[x] = u; }
  void set_slack(int x) {
    slack[x] = 0;
    for (int u = 1; u <= n; ++u)
      if (g[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
        update_slack(u, x);
  }
  void q_push(int x) {
    if (x <= n) q.push(x);
    else for (size_t i = 0; i < flo[x].size(); i++) q_push(flo[x][i]);
  }
  void set_st(int x, int b) {
    st[x] = b;
    if (x > n) for (size_t i = 0; i < flo[x].size(); ++i)
      set_st(flo[x][i], b);
  }
  int get_pr(int b, int xr) {
    int pr = find(flo[b].begin(), flo[b].end(), xr) - flo[b].begin();
    if (pr % 2 == 1) {
      reverse(flo[b].begin() + 1, flo[b].end());
      return (int)flo[b].size() - pr;
    }
    return pr;
  }
  void set_match(int u, int v) {
    match[u] = g[u][v].v;
    if (u <= n) return;
    edge e = g[u][v];
    int xr = flo_from[u][e.u], pr = get_pr(u, xr);
```

```
    for (int i = 0; i < pr; ++i) set_match(flo[u][i], flo[u][i ^ 1]);
    set_match(xr, v);
    rotate(flo[u].begin(), flo[u].begin() + pr, flo[u].end());
  }
  void augment(int u, int v) {
    for (; ; ) {
      int xnv = st[match[u]];
      set_match(u, v);
      if (!xnv) return;
      set_match(xnv, st[pa[xnv]]);
      u = st[pa[xnv]], v = xnv;
    }
  }
  int get_lca(int u, int v) {
    static int t = 0;
    for (++t; u || v; swap(u, v)) {
      if (u == 0) continue;
      if (vis[u] == t) return u;
      vis[u] = t;
      u = st[match[u]];
      if (u) u = st[pa[u]];
    }
    return 0;
  }
  void add_blossom(int u, int lca, int v) {
    int b = n + 1;
    while (b <= n_x && st[b]) ++b;
    if (b > n_x) ++n_x;
    lab[b] = 0, S[b] = 0;
    match[b] = match[lca];
    flo[b].clear();
    flo[b].push_back(lca);
    for (int x = u, y; x != lca; x = st[pa[y]])
      flo[b].push_back(x), flo[b].push_back(y = st[match[x]]), q_push(y);
    reverse(flo[b].begin() + 1, flo[b].end());
    for (int x = v, y; x != lca; x = st[pa[y]])
      flo[b].push_back(x), flo[b].push_back(y = st[match[x]]), q_push(y);
    set_st(b, b);
    for (int x = 1; x <= n_x; ++x) g[b][x].w = g[x][b].w = 0;
    for (int x = 1; x <= n; ++x) flo_from[b][x] = 0;
    for (size_t i = 0; i < flo[b].size(); ++i) {
      int xs = flo[b][i];
      for (int x = 1; x <= n_x; ++x)
        if (g[b][x].w == 0 || e_delta(g[xs][x]) < e_delta(g[b][x]))
          g[b][x] = g[xs][x], g[x][b] = g[x][xs];
      for (int x = 1; x <= n; ++x)
        if (flo_from[xs][x]) flo_from[b][x] = xs;
    }
    set_slack(b);
  }
  void expand_blossom(int b) {
    for (size_t i = 0; i < flo[b].size(); ++i)
      set_st(flo[b][i], flo[b][i]);
    int xr = flo_from[b][g[b][pa[b]].u], pr = get_pr(b, xr);
    for (int i = 0; i < pr; i += 2) {
      int xs = flo[b][i], xns = flo[b][i + 1];
      pa[xs] = g[xns][xs].u;
      S[xs] = 1, S[xns] = 0;
      slack[xs] = 0, set_slack(xns);
      q_push(xns);
    }
    S[xr] = 1, pa[xr] = pa[b];
    for (size_t i = pr + 1; i < flo[b].size(); ++i) {
      int xs = flo[b][i];
      S[xs] = -1, set_slack(xs);
    }
    st[b] = 0;
  }
  bool on_found_edge(const edge &e) {
    int u = st[e.u], v = st[e.v];
    if (S[v] == -1) {
      pa[v] = e.u, S[v] = 1;
      int nu = st[match[v]];
      slack[v] = slack[nu] = 0;
      S[nu] = 0, q_push(nu);
    } else if (S[v] == 0) {
      int lca = get_lca(u, v);
      if (!lca)
        return augment(u,v), augment(v,u), true;
      else
        add_blossom(u, lca, v);
    }
    return false;
  }
  bool matching() {
    memset(S + 1, -1, sizeof(int) * n_x);
    memset(slack + 1, 0, sizeof(int) * n_x);
    q = queue<int>();
    for (int x = 1; x <= n_x; ++x)
      if (st[x] == x && !match[x])
        pa[x] = 0, S[x] = 0, q_push(x);
    if (q.empty()) return false;
    for (; ; ) {
      while (q.size()) {
        int u = q.front(); q.pop();
        if (S[st[u]] == 1) continue;
        for (int v = 1; v <= n; ++v)
          if (g[u][v].w > 0 && st[u] != st[v]) {
            if (e_delta(g[u][v]) == 0) {
              if (on_found_edge(g[u][v]))
                return true;
            } else
              update_slack(u, st[v]);
          }
      }
      int d = inf;
      for (int b = n + 1; b <= n_x; ++b)
        if (st[b] == b && S[b] == 1) d = min(d, lab[b] / 2);
      for (int x = 1; x <= n_x; ++x)
        if (st[x] == x && slack[x]) {
          if (S[x] == -1) d = min(d, e_delta(g[slack[x]][x]));
          else if (S[x] == 0) d = min(d, e_delta(g[slack[x]][x]) / 2);
        }
      for (int u = 1; u <= n; ++u) {
        if (S[st[u]] == 0) {
          if (lab[u] <= d) return 0;
          lab[u] -= d;
        } else if (S[st[u]] == 1) lab[u] += d;
      }
      for (int b = n + 1; b <= n_x; ++b)
        if (st[b] == b) {
          if (S[st[b]] == 0) lab[b] += d * 2;
          else if (S[st[b]] == 1) lab[b] -= d * 2;
        }
      q = queue<int>();
      for (int x = 1; x <= n_x; ++x)
        if (st[x] == x && slack[x] && st[slack[x]] != x && e_delta(g[slack[x]][x]) == 0)
          if (on_found_edge(g[slack[x]][x]))
            return true;
      for (int b = n + 1; b <= n_x; ++b)
        if (st[b] == b && S[b] == 1 && lab[b] == 0)
          expand_blossom(b);
    }
    return false;
  }
  pair<long long, int> solve() {
    memset(match + 1, 0, sizeof(int) * n);
    n_x = n;
    int n_matches = 0;
    long long tot_weight = 0;
    for (int u = 0; u <= n; ++u) st[u] = u, flo[u].clear();
    int w_max = 0;
    for (int u = 1; u <= n; ++u)
      for (int v = 1; v <= n; ++v) {
        flo_from[u][v] = (u == v ? u : 0);
        w_max = max(w_max, g[u][v].w);
      }
    for (int u = 1; u <= n; ++u) lab[u] = w_max;
    while (matching()) ++n_matches;
    for (int u = 1; u <= n; ++u)
      if (match[u] && match[u] < u)
        tot_weight += g[u][match[u]].w;
    return make_pair(tot_weight, n_matches);
  }
  void add_edge(int ui, int vi, int wi) {
    g[ui][vi].w = g[vi][ui].w = wi;
  }
  void init(int _n) {
    n = _n;
    for (int u = 1; u <= n; ++u)
      for (int v = 1; v <= n; ++v)
        g[u][v] = edge(u, v, 0);
  }
};
```

3.6 Minimum Cost Maximum Flow

```
struct Flow {
  struct Edge {
    int u, rc, k, rv;
  };
  vector<vector<Edge>> G;
  vector<int> par, par_eid;
  Flow(int n) : G(n + 1), par(n + 1), par_eid(n + 1) {}
  void add(int v, int u, int c, int k) {
    G[v].push_back({u, c, k, (int)G[u].size()});
    G[u].push_back({v, 0, -k, (int)G[v].size() - 1});
  }
  int spfa(int s, int t) {
    fill(par.begin(), par.end(), -1);
    vector<int> dis(par.size(), LONG_LONG_MAX);
    vector<bool> in_q(par.size(), false);
    queue<int> Q;
    dis[s] = 0; in_q[s] = true;
    Q.push(s);
```

```
    while (! Q.empty()) {
      int v = Q.front(); Q.pop();
      in_q[v] = false;
      for (int i = 0; i < (int)G[v].size(); i++) {
        auto [u, rc, k, rv] = G[v][i];
        if (rc > 0 && dis[v] + k < dis[u]) {
          dis[u] = dis[v] + k;
          par[u] = v;
          par_eid[u] = i;
          if (! in_q[u]) Q.push(u);
          in_q[u] = true;
        }
      }
    }
    return dis[t];
  }
  pair<int, int> flow(int s, int t) {
    int fl = 0, cost = 0, d;
    while ((d = spfa(s, t)) < LONG_LONG_MAX) {
      int cur = LONG_LONG_MAX;
      for (int v = t; v != s; v = par[v])
        cur = min(cur, G[par[v]][par_eid[v]].rc);
      fl += cur;
      cost += d * cur;
      for (int v = t; v != s; v = par[v]) {
        G[par[v]][par_eid[v]].rc -= cur;
        G[v][G[par[v]][par_eid[v]].rv].rc += cur;
      }
    }
    return {fl, cost};
  }
};
```

4 geometry

4.1 basic

```
const double eps = 1e-8, PI = acos(-1);
int sign(double x)
{ return fabs(x) <= eps ? 0 : (x > 0 ? 1 : -1); }
double normalize(double x) {
  while (x < -eps) x += PI * 2;
  while (x > PI * 2 + eps) x -= PI * 2;
  return x; }
template <typename T> struct P {
  T x, y;
  P (T _x = 0, T _y = 0) : x(_x), y(_y) {}
  P<T> operator + (P<T> o) {
    return P<T>(x + o.x, y + o.y);}
  P<T> operator - (P<T> o) {
    return P<T>(x - o.x, y - o.y);}
  P<T> operator * (T k) {return P<T>(x * k, y * k);}
  P<T> operator / (T k) {return P<T>(x / k, y / k);}
  T operator * (P<T> o) {return x * o.x + y * o.y;}
  T operator ^ (P<T> o) {return x * o.y - y * o.x;}
  friend ostream& operator << (ostream &o, P<T> a) {
    return o << "(" << a.x << ", " << a.y << ")"; }
  bool operator == (P<T> o) {
    return sign(x - o.x) == 0 && sign(y - o.y) == 0;}
};
using Pt = P<ll>;
struct Line { Pt a, b; };
struct Cir { Pt o; double r; };
ll abs2(Pt a) { return a * a; }
double abs(Pt a) { return sqrt(abs2(a)); }
int ori(Pt o, Pt a, Pt b)
{ return sign((o - a) ^ (o - b)); }
bool btw(Pt a, Pt b, Pt c) // c on segment ab?
{ return ori(a, b, c) == 0 &&
         sign((c - a) * (c - b)) <= 0; }
int pos(Pt a)
{ return sign(a.y) == 0 ? sign(a.x) < 0 : a.y < 0; }
bool cmp(Pt a, Pt b)
{ return pos(a) == pos(b) ? sign(a ^ b) > 0 :
         pos(a) < pos(b); }
bool same_vec(Pt a, Pt b, int d) // d = 1: check dir
{ return sign(a ^ b) == 0 && sign(a * b) > d * 2 - 2; }
bool same_vec(Line a, Line b, int d)
{ return same_vec(a.b - a.a, b.b - b.a, d); }
Pt perp(Pt a) { return Pt(-a.y, a.x); } // CCW 90 deg
Pt ref(Pt a) {return pos(a) == 1 ? Pt(-a.x, -a.y) : a;}
// double part
double theta(Pt a)
{ return normalize(atan2(a.y, a.x)); }
Pt unit(Pt o) { return o / abs(o); }
Pt rot(Pt a, double o) // CCW
{ double c = cos(o), s = sin(o);
  return Pt(c * a.x - s * a.y, s * a.x + c * a.y); }
Pt proj_vec(Pt a, Pt b, Pt c) // vector ac proj to ab
{return (b - a) * ((c - a) * (b - a)) / (abs2(b - a));}
Pt proj_pt(Pt a, Pt b, Pt c) // point c proj to ab
{ return proj_vec(a, b, c) + a; }
```

4.2 convex hull

```
auto convex_hull(vector<Pt> pts) {
  sort(all(pts), [&](Pt a, Pt b)
  { return a.x == b.x ? a.y < b.y : a.x < b.x; });
  vector<Pt> ans = {pts[0]};
  for (int t = 0; t < 2; ++t, reverse(all(pts))) {
    for (int i = 1, m = sz(ans); i < sz(pts); ++i) {
      while (sz(ans) > m && ori(ans[sz(ans) - 2],
        ans.back(), pts[i]) <= 0) ans.pop_back();
      ans.pb(pts[i]);
    }
  }
  if (sz(ans) > 1) ans.pop_back();
  return ans;
}
```

4.3 convex hull dist

```
// 計算兩個凸多邊形 A、B 之間的最短距離
// 兩者相交或接觸就回傳 0，否則回傳兩者邊界上最近兩點的距離
double convex_hull_dist(vector<Pt> A, vector<Pt> B) {
  Pt O(0, 0); for (auto &p : B) p = O - p;
  auto C = minkowski(A, B); // assert SZ(C) > 0
  if (point_in_convex(C, O)) return 0;
  double ans = point_segment_dist(C.back(), C[0], O);
  for (int i = 0; i + 1 < sz(C); ++i)
    ans = min(ans, point_segment_dist(C[i], C[i + 1], O));
  return ans;
}
```

4.4 Minimum Enclosing Circle

```
Cir min_enclosing(vector<Pt> p) {
  random_shuffle(all(p));
  double r = 0.0;
  Pt cent = p[0];
  for (int i = 1; i < sz(p); ++i) {
    if (abs2(cent - p[i]) <= r) continue;
    cent = p[i], r = 0.0;
    for (int j = 0; j < i; ++j) {
      if (abs2(cent - p[j]) <= r) continue;
      cent = (p[i] + p[j]) / 2, r = abs2(p[j] - cent);
      for (int k = 0; k < j; ++k) {
        if (abs2(cent - p[k]) <= r) continue;
        cent = circenter(p[i], p[j], p[k]);
        r = abs2(p[k] - cent);
      }
    }
  }
  return {cent, sqrt(r)};
}
```

4.5 Minkowski

```
auto minkowski(vector<Pt> a, vector<Pt> b) {
  a = convex_hull(a), b = convex_hull(b);
  int n = sz(a), m = sz(b);
  vector<Pt> c = {a[0] + b[0]}, s1, s2;
  for(int i = 0; i < n; ++i)
    s1.pb(a[(i + 1) % n] - a[i]);
  for(int i = 0; i < m; i++)
    s2.pb(b[(i + 1) % m] - b[i]);
  for(int p1 = 0, p2 = 0; p1 < n || p2 < m;)
    if (p2 == m || (p1 < n && sign(s1[p1] ^ s2[p2]) >= 0))
      c.pb(c.back() + s1[p1++]);
    else
      c.pb(c.back() + s2[p2++]);
  return convex_hull(c);
}
```

4.6 Point In Convex

```
// strict: if true, point on edge is considered outside
bool point_in_convex(vector<Pt> &C, Pt p, bool strict = true) {
  // only works when no three points are collinear
  int a = 1, b = sz(C) - 1, r = !strict;
  if (sz(C) == 0) return false;
  if (sz(C) < 3) return r && btw(C[0], C.back(), p);
  if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
  if (ori(C[0], C[a], p) >= r || ori(C[0], C[b], p) <= -r)
      return false;
```

```
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (ori(C[0], C[c], p) > 0 ? b : a) = c;
  }
  return ori(C[a], C[b], p) < r;
}
// 回傳 p 所在的邊 {i, j}
// p 不在邊界上時回傳 {-1, -1}
pair<int, int> point_on_convex_edge(vector<Pt> &C, Pt p) {
  int a = 1, b = sz(C) - 1;
  if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
  if (ori(C[0], C[a], p) == 0 && btw(C[0], C[a], p)) return {0, a};
  if (ori(C[0], C[b], p) == 0 && btw(C[0], C[b], p)) return {0, b};
  if (ori(C[0], C[a], p) > 0 || ori(C[0], C[b], p) < 0) return {-1, -1};
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (ori(C[0], C[c], p) > 0 ? b : a) = c;
  }
  if (ori(C[a], C[b], p) == 0) return {a, b};
  return {-1, -1};
}
```

4.7 Point In Poly

```
// 判斷點是否在多邊形內
// -1: 在外面
//  0: 在邊上（只有 strict = 1 時會出現）
//  1: 在裡面 (strict = 0 時，在邊上也回傳 1)
// -1: out, 0: edge, 1: in
int point_in_poly(vector <Pt> poly, Pt o, int strict) {
  int cnt = 0;
  for (int i = 0; i < sz(poly); ++i) {
    Pt a = poly[i], b = poly[(i + 1) % sz(poly)];
    if (btw(o, a, b)) return !strict;
    cnt ^= ((o.y < a.y) - (o.y - b.y)) * ori(o, a, b) > 0;
  }
  return cnt ? 1 : -1;
}
```

4.8 Point In Triangle

```
// S: 點集（排序、去重）
// dirs[i]: 所有 p > S[i] 的向量 p - S[i]，依極角排序
// 這些向量都落在半平面 (x > 0 或 x == 0 && y > 0)，所以用外積比較是合法的全序
vector<Pt> S;
vector<vector<Pt>> dirs;
int id(Pt p) { return lower_bound(all(S), p) - S.begin(); }
void init(vector<Pt> pts) {
  S = pts; sort(all(S)); S.erase(unique(all(S)), S.end());
  int n = sz(S); dirs.assign(n, {});
  for (int i = 0; i < n; ++i) {
    for (int j = i + 1; j < n; ++j) dirs[i].push_back(S[j] - S[i]);
    sort(all(dirs[i]), [](Pt u, Pt v) { return (u ^ v) > 0; });
  }
}
// q 必須在 S 裡；在所有 p > q 中，數 (d ^ (p - q)) < 0 與 == 0 的個數
pair<int, int> cnt(Pt q, Pt d) {
  auto &v = dirs[id(q)];
  auto cmp = [](Pt u, Pt w) { return (u ^ w) > 0; };
  auto lo = lower_bound(all(v), d, cmp), hi = upper_bound(all(v), d, cmp);
  return {int(lo - v.begin()), int(hi - lo)};
}
// check if this number is calculated（p 本身是不是點集裡的點）
bool check(Pt p) { int i = id(p); return i < sz(S) && S[i] == p; }
// number of points p with a < p < b such that ori(p, a, b) < 0
int under(Pt a, Pt b) {
  Pt d = b - a;
  return cnt(a, d).first - cnt(b, d).first;
}
// number of points with a < p < b and ori(p, a, b) = 0
int edge(Pt a, Pt b) {
  Pt d = b - a;
  return cnt(a, d).second - cnt(b, d).second - check(b);
}
// number of points that strictly inside the triangle
int in_tri(array<Pt, 3> arr) {
  sort(all(arr), [&](Pt i, Pt j) {
    return i.x == j.x ? i.y < j.y : i.x < j.x; });
  auto [a, b, c] = arr;
  int x = ori(b, a, c);
  if (x == 0) return 0;
  if (x == 1) return under(a, b) + under(b, c) - under(a, c) - edge(a, c);
  return under(a, c) - under(a, b) - under(b, c) - edge(a, b) - edge(b, c) - check(b);
}
int main() {
  int n; cin >> n;
  vector<Pt> pts(n);
  for (auto &p : pts) cin >> p.x >> p.y;
  init(pts);
  int q; cin >> q;
  while (q--) {
    int i, j, k; cin >> i >> j >> k;  // 三個頂點在輸入中的編號 (0-indexed)
    cout << in_tri({pts[i], pts[j], pts[k]}) << '\n';
  }
}
```

4.9 Point Segment Dist

```
double point_segment_dist(Pt q0, Pt q1, Pt p) {
  if (sign(abs(q0 - q1)) == 0) return abs(q0 - p);
  if (sign((q1 - q0) * (p - q0)) >= 0 && sign((q0 - q1) * (p - q1)) >= 0)
    return fabs(((q1 - q0) ^ (p - q0)) / abs(q0 - q1));
  return min(abs(p - q0), abs(p - q1));
}
```

5 graph

5.1 2SAT

```
struct TWO_SAT {
  int n, N;
  vector<vector<int>> G, rev_G;
  deque<bool> used;
  vector<int> order, comp;
  deque<bool> assignment;
  void init(int _n) {
    n = _n;
    N = _n * 2;
    G.resize(N + 5);
    rev_G.resize(N + 5);
  }
  void dfs1(int v) {
    used[v] = true;
    for (int u : G[v]) {
      if (!used[u])
        dfs1(u);
    }
    order.push_back(v);
  }
  void dfs2(int v, int cl) {
    comp[v] = cl;
    for (int u : rev_G[v]) {
      if (comp[u] == -1)
        dfs2(u, cl);
    }
  }
  bool solve() {
    order.clear();
    used.assign(N, false);
    for (int i = 0; i < N; ++i) {
      if (!used[i])
        dfs1(i);
    }
    comp.assign(N, -1);
    for (int i = 0, j = 0; i < N; ++i) {
      int v = order[N - i - 1];
      if (comp[v] == -1)
        dfs2(v, j++);
    }
    assignment.assign(n, false);
    for (int i = 0; i < N; i += 2) {
      if (comp[i] == comp[i + 1])
        return false;
      assignment[i / 2] = (comp[i] > comp[i + 1]);
    }
    return true;
  }
  void add_disjunction(int a, bool na, int b, bool nb) { // A or B
    // na means whether a is negative or not
    // nb means whether b is negative or not
    a = 2 * a ^ na;
    b = 2 * b ^ nb;
    int neg_a = a ^ 1;
    int neg_b = b ^ 1;
```

```
    G[neg_a].push_back(b);
    G[neg_b].push_back(a);
    rev_G[b].push_back(neg_a);
    rev_G[a].push_back(neg_b);
    return;
  }
  void get_result(vector<int>& res) {
    res.clear();
    for (int i = 0; i < n; i++)
      res.push_back(assignment[i]);
  }
};
```

5.2 bridge

```
const int N = 200005;
vector<int> G[N];
int low[N], depth[N];
bool vis[N];
vector<pair<int, int>> bridge;
void dfs(int v, int p) {
  vis[v] = true;
  low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
  for (int u : G[v]) {
    if (u == p) continue;
    if (!vis[u]) {
      // (v, u) 是樹邊
      dfs(u, v);
      low[v] = min(low[v], low[u]);
    } else {
      // (v, u) 是回邊
      low[v] = min(low[v], depth[u]);
    }
  }
  // v 在不依靠父邊的情況下永遠沒辦法走到它的祖先
  // (root, -1) 不算
  if (low[v] == depth[v] && p != -1)
    bridge.push_back({v, p});
}
```

5.3 bridge bcc

```
const int N = 200005;
vector<int> G[N];
int low[N], depth[N];
bool vis[N];
vector<vector<int>> bcc;
stack<int> stk;
void dfs(int v, int p) {
  stk.push(v);
  vis[v] = true;
  low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
  for (int u : G[v]) {
    if (u == p) continue;
    if (!vis[u]) {
      // (v, u) 是樹邊
      dfs(u, v);
      low[v] = min(low[v], low[u]);
    } else {
      // (v, u) 是回邊
      low[v] = min(low[v], depth[u]);
    }
  }
  // v 在不依靠父邊的情況下永遠沒辦法走到它的祖先
  if (low[v] == depth[v]) {
    bcc.emplace_back();
    while (stk.top() != v) {
      bcc.back().push_back(stk.top());
      stk.pop();
    }
    bcc.back().push_back(stk.top());
    stk.pop();
  }
}
```

5.4 cut

```
const int N = 200005;
vector<int> G[N];
int low[N], depth[N];
bool vis[N], cut[N];
void dfs(int v, int p) {
  vis[v] = true;
  int child = 0;
  low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
  for (int u : G[v]) {
    if (u == p) continue;
    if (!vis[u]) {
      // (v, u) 是樹邊
      dfs(u, v);
      child++;
      low[v] = min(low[v], low[u]);
      // u 無法在不經過父邊的情況走到 v 的祖先
      if (low[u] >= depth[v] && p != -1)
        cut[v] = true;
    } else {
      // (v, u) 是回邊
      low[v] = min(low[v], depth[u]);
    }
  }
  // 根節點有超過 2 個子節點
  if (p == -1 && child >= 2)
    cut[v] = true;
}
```

5.5 cut bcc

```
const int N = 200005;
vector<int> G[N];
int low[N], depth[N];
bool vis[N];
vector<vector <int>> bcc;
stack <int> stk;
void dfs(int v, int p) {
  stk.push(v);
  vis[v] = true;
  low[v] = depth[v] = (p == -1 ? 1 : depth[p] + 1);
  for (int u : G[v]) {
    if (u == p) continue;
    if (!vis[u]) {
      // (v, u) 是樹邊
      dfs(u, v);
      low[v] = min(low[v], low[u]);
      // u 無法在不經過父邊的情況走到 v 的祖先
      if (low[u] >= depth[v]) {
        bcc.emplace_back();
        while (stk.top() != u) {
          bcc.back().push_back(stk.top());
          stk.pop();
        }
        bcc.back().push_back(stk.top());
        stk.pop();
        bcc.back().push_back(v);
      }
    } else {
      // (v, u) 是回邊
      low[v] = min(low[v], depth[u]);
    }
  }
}
```

5.6 prim

```
const int N = 2005;
int dis[N][N];
int distfromtree[N];
for (int i = 1; i <= n; i++)
  distfromtree[i] = dis[1][i];
int res = 0;
for (int i = 2; i <= n; i++) {
  int mindist = INT_MAX, next = -1;
  for (int j = 1; j <= n; j++) {
    if (distfromtree[j] != 0 && distfromtree[j] < mindist) {
      mindist = distfromtree[j];
      next = j;
    }
  }
  res += mindist;
  for (int j = 1; j <= n; j++)
    distfromtree[j] = min(distfromtree[j], dis[next][j]);
}
```

5.7 spfa

```
vector<int> G[N];
int spfa(int n, int s, int t) {
  vector<int> dis(n, INT_MAX), cnt(n);
  vector<bool> inq(n);
  queue<int> q;
  q.push(s);
  dis[s] = 0;
  inq[s] = true;
```

```
  while (q.size()) {
    int u = q.front();
    q.pop();
    cnt[u]++;
    if (cnt[u] == n) // negative cycle
      return INT_MAX;
    inq[u] = false;
    for (auto [v, w] : G[u]) {
      if (dis[u] + w < dis[v]) {
        dis[v] = dis[u] + w;
        if (!inq[v]) {
          inq[v] = true;
          q.push(v);
        }
      }
    }
  }
  return dis[t];
}
```

5.8 Strongly Connected Component

```
const int N = 200005;
vector<int> G[N], rev_G[N], order;
bool vis[N]; int scc[N];
void rev_dfs(int v) {
  vis[v] = true;
  for (int u : rev_G[v])
    if (!vis[u])
      rev_dfs(u);
  order.push_back(v);
}
void dfs(int v, int block) {
  scc[v] = block;
  for (int u : G[v])
    if (scc[u] == -1)
      dfs(u, block);
}
void kosaraju(int n) {
  for (int i = 1; i <= n; i++)
    if (!vis[i])
      rev_dfs(i);
  int scc_cnt = 0;
  fill(scc, scc + N, -1);
  for (int i = n - 1; i >= 0; i--)
    if (scc[order[i]] == -1)
      dfs(order[i], ++scc_cnt);
}
```

6 math

6.1 big number

```
void init(string &a, string &b) {
  while (a.size() < b.size()) a = '0' + a;
  while (a.size() > b.size()) b = '0' + b;
}
int Compare(string &a, string &b) {
  if (a < b) {
    swap(a, b); return -1;
  }
  return 1;
}
bool del(string &a) {
  if (a[0] == '0') {
    a.erase(0, 1); return true;
  }
  return false;
}
void del_all_zero(string &a) {
  while (del(a)) ;
}
string add(string a, string b) {
  init(a, b);
  a = '0' + a; b = '0' + b;
  for (int i = a.size() - 1; i >= 0; i--) {
    int p = a[i] - '0', q = b[i] - '0';
    if (p + q >= 10) {
      a[i - 1] = '0' + (a[i - 1] - '0' + 1);
      a[i] = '0' + (p + q - 10);
    } else {
      a[i] = '0' + (p + q);
    }
  }
  del(a);
  return a;
}
string sub(string a, string b) {
  init(a, b);
  if (a == b) return "0";
  int flag = Compare(a, b);
  for (int i = a.size() - 1; i >= 0; i--) {
    int p = a[i] - '0', q = b[i] - '0';
    if (p < q) {
      a[i - 1] = '0' + (a[i - 1] - '0' - 1);
      a[i] = '0' + (p - q + 10);
    } else {
      a[i] = '0' + (p - q);
    }
  }
  del_all_zero(a);
  if (flag == -1) a = "-" + a;
  return a;
}
string mul(string a, string b) {
  string res = "0";
  init(a, b);
  Compare(a, b);
  del_all_zero(b);
  for (int i = b.size() - 1; i >= 0; i--) {
    int x = b[i] - '0';
    if (i != b.size() - 1) a = a + '0';
    for (int i = 0; i < x; i++)
      res = add(a, res);
  }
  return res;
}
string div(string a, string b) {
  init(a, b);
  if (a < b) return "0";
  del_all_zero(b);
  string res = "0", restmp = "1", tmp = b;
  for (int i = 0; i < (a.size() - b.size()); i++) {
    restmp += '0';
    tmp += '0';
  }
  init(a, b);
  while (a >= b) {
    init(a, tmp);
    if (a >= tmp) {
      a = sub(a, tmp);
      res = add(res, restmp);
    } else {
      restmp.erase(restmp.size() - 1);
      tmp.erase(tmp.size() - 1);
    }
    init(a, b);
  }
  return res;
}
```

6.2 burnside lemma

$$\sum_{k=1}^{n} \frac{c(k)}{n}$$

- n：有多少種置換方式 (例如：旋轉方式)
- $c(k)$：所有可能中，經過 k 次旋轉後，仍不會和別人相同方式的數量

6.3 catalan

$C_n = \frac{1}{n+1}\binom{2n}{n}$

6.4 Chinese Remainder Theorem coprime

```
vector<int> a, m;
int extgcd(int a, int b, int& x, int& y) {
  if (b == 0) {
    x = 1, y = 0;
    return a;
  }
  int ret = extgcd(b, a % b, y, x);
  y -= a / b * x;
  return ret;
}
// n = 有幾個式子，求解 x \equiv a_i \bmod m_i
int CRT(int n, vector<int>& a, vector<int>& m) {
  int p = 1, ans = 0;
  vector<int> M(n), inv_M(n);
  for (int i = 0; i < n; i++)
    p *= m[i];
  for (int i = 0; i < n; i++) {
    M[i] = p / m[i];
    extgcd(M[i], m[i], inv_M[i], tmp);
    ans += a[i] * inv_M[i] * M[i];
    ans %= p;
  }
```

```
  return (ans % p + p) % p;
}
```

6.5 Chinese Remainder Theorem coprime

$n =$ 有幾個式子，求解 $x \equiv a_i \bmod m_i$

6.6 Chinese Remainder Theorem not coprime

```
int extgcd(int a, int b, int& x, int& y) {
  if (b == 0) {
    x = 1, y = 0;
    return a;
  }
  int ret = extgcd(b, a % b, y, x);
  y -= a / b * x;
  return ret;
}
// 對於方程組的式子兩兩求解， return {是否有解, {a, m}}
pair<bool, pair<int, int>> CRT(int a1, int m1, int a2, int m2) {
  int g = __gcd(m1, m2);
  if ((a2 - a1) % g != 0)
    return {0, {-1, -1}};
  int x, y;
  extgcd(m1, m2, x, y);
  x = (a2 - a1) * x / g; // 兩者不能相反
  a1 = x * m1 + a1;
  m1 = m1 * m2 / g;
  a1 = (a1 % m1 + m1) % m1;
  return {1, {a1, m1}};
}
```

6.7 count prime

```
// Count the number of primes not greater than N (N <= 10^11)
int primeCount(const int n) {
  if (n <= 1) { return 0; }
  if (n == 2) { return 1; }
  const int v = sqrtl(n);
  int s = (v + 1) / 2;
  vector<int> smalls(s), roughs(s), skip(v + 1), larges(s);
  iota(smalls.begin(), smalls.end(), 0);
  for (int i = 0; i < s; i++) {
    roughs[i] = 2 * i + 1;
    larges[i] = (n / roughs[i] - 1) / 2;
  }
  const auto half = [](int n) -> int { return (n - 1) >> 1; };
  int pc = 0;
  for (int p = 3; p <= v; p += 2) {
    if (skip[p]) { continue; }
    int q = p * p;
    if (1LL * q * q > n) { break; }
    skip[p] = true;
    for (int i = q; i <= v; i += 2 * p) skip[i] = true;
    int ns = 0;
    for (int k = 0; k < s; k++) {
      int i = roughs[k];
      if (skip[i]) { continue; }
      int d = 1LL * i * p;
      larges[ns] = larges[k] - (d <= v ? larges[smalls[d / 2] -
          pc] : smalls[half(n / d)]) + pc;
      roughs[ns++] = i;
    }
    s = ns;
    for (int i = half(v), j = v / p - 1 | 1; j >= p; j -= 2) {
      int c = smalls[j / 2] - pc;
      for (int e = j * p / 2; i >= e; i--) { smalls[i] -= c; }
    }
    pc++;
  }
  larges[0] += 1LL * (s + 2 * (pc - 1)) * (s - 1) / 2;
  for (int k = 1; k < s; k++) { larges[0] -= larges[k]; }
  for (int l = 1; l < s; l++) {
    int q = roughs[l];
    int M = n / q;
    int e = smalls[half(M / q)] - pc;
    if (e <= l) { break; }
    int t = 0;
    for (int k = l + 1; k <= e; k++) { t += smalls[half(M /
        roughs[k])]; }
    larges[0] += t - 1LL * (e - l) * (pc + l - 1);
  }
  return larges[0] + 1;
}
```

6.8 det

```
ll Det(vector <vector <ll>> a) {
  int n = a.size();
  ll det = 1;
  for (int i = 0; i < n; ++i) {
    if (!a[i][i]) {
      det = -det;
      if (det < 0) det += mod;
      for (int j = i + 1; j < n; ++j) if (a[j][i]) {
        swap(a[j], a[i]);
        break;
      }
      if (!a[i][i]) return 0;
    }
    det = det * a[i][i] % mod;
    ll mul = mpow(a[i][i], mod - 2);
    for (int j = 0; j < n; ++j)
      a[i][j] = a[i][j] * mul % mod;
    for (int j = 0; j < n; ++j) if (i ^ j) {
      ll mul = a[j][i];
      for (int k = 0; k < n; ++k) {
        a[j][k] -= a[i][k] * mul % mod;
        if (a[j][k] < 0) a[j][k] += mod;
      }
    }
  }
  return det;
}
```

6.9 extgcd

```
// a * p.first + b * p.second = gcd(a, b)
pair<ll, ll> extgcd(ll a, ll b) {
  pair<ll, ll> res, tmp;
  ll f = 1, g = 1;
  if (a < 0) a *= -1, f *= -1;
  if (b < 0) b *= -1, g *= -1;
  if (b == 0) return {f, 0};
  tmp = extgcd(b, a % b);
  res.first = tmp.second * f;
  res.second = (tmp.first - tmp.second * (a / b)) * g;
  return res;
}
```

6.10 factorize

```
using i64 = long long;
using i128 = __int128_t;
struct Factorize { // 質因數分解 N <= 10^18
  i64 fmul(i64 a, i64 b, i64 p) {
    return (i128)a * b % p;
  }
  i64 fpow(i64 a, i64 b, i64 p) {
    i64 res = 1;
    for (; b; b >>= 1, a = fmul(a, a, p))
      if (b & 1) res = fmul(res, a, p);
    return res;
  }
  bool check(i64 a, i64 u, i64 n, int t) {
    a = fpow(a, u, n);
    if (a == 0 || a == 1 || a == n - 1)
      return true;
    for (int i = 0; i < t; i++) {
      a = fmul(a, a, n);
      if (a == 1) return false;
      if (a == n - 1) return true;
    }
    return false;
  };
  bool isPrime(i64 n) {
    constexpr array<i64, 7> magic{2, 235, 9375, 28178, 450775,
        9780504, 1795265022};
    // for int: {2, 7, 61}
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    i64 u = n - 1;
    int t = 0;
    while (u % 2 == 0) u >>= 1, t++;
    for (auto v : magic) if (!check(v, u, n, t)) return false;
    return true;
  }
  i64 PollardRho(i64 n) { // return non-trivial factor of n
    if (n % 2 == 0) return 2;
    i64 x = 2, y = 2, d = 1, p = 1;
    auto f = [](i64 x, i64 n, i64 p) -> i64 {
      return ((i128)x * x % n + p) % n;
    };
    while (true) {
```

```
      x = f(x, n, p);
      y = f(f(y, n, p), n, p);
      d = gcd(abs(x - y), n);
      if (d != n && d != 1) return d;
      if (d == n) ++p;
    }
  }
  i64 primeFactor(i64 n) {
    return isPrime(n) ? n : primeFactor(PollardRho(n));
  }
} ftr;
void solve() {
  i64 n;
  cin >> n;
  vector<i64> ans;
  while (n > 1) {
    i64 p = ftr.primeFactor(n);
    do {
      ans.push_back(p);
      n /= p;
    } while (n % p == 0);
  }
  sort(ans.begin(), ans.end());
  cout << ans.size() << ' ';
  for (int i : ans) cout << i << ' ';
  cout << '\n';
}
```

6.11 fraction

```
struct frac {
  int a, b;
  frac(int _a = 0, int _b = 1) : a(_a), b(_b) {
    int g = __gcd(a, b);
    a /= g; b /= g;
    if (b < 0) { a *= -1; b *= -1; }
  }
};
frac operator+(frac x, frac y) { return frac(x.a * y.b + y.a * x.b, x.b * y.b); }
frac operator-(frac x, frac y) { return frac(x.a * y.b - y.a * x.b, x.b * y.b); }
frac operator*(frac x, frac y) { return frac(x.a * y.a, x.b * y.b); }
frac operator/(frac x, frac y) { return frac(x.a * y.b, x.b * y.a); }
bool operator>(frac x, frac y) { return x.a * y.b > y.a * x.b; }
bool operator<(frac x, frac y) { return x.a * y.b < y.a * x.b; }
bool operator>=(frac x, frac y) { return x.a * y.b >= y.a * x.b; }
bool operator<=(frac x, frac y) { return x.a * y.b <= y.a * x.b; }
bool operator==(frac x, frac y) { return x.a * y.b == y.a * x.b; }
frac abs(frac x) { return frac(abs(x.a), abs(x.b)); }
ostream& operator<<(ostream &os, const frac& x) { os << x.a << "/" << x.b; return os; }
```

6.12 gaussian elimination

```
#define int long long
const int N = 25;
frac co[N][N];
// SIZE = n * (n + 1)
// a_1 x_1 + a_2 x_2 + ... + a_n * x_n = a_{n + 1}
void gaussian(int n) {
  for (int i = 1; i <= n; i++) {
    if (co[i][i].a == 0 && co[i][i].b == 1) { // swap
      int swapid = -1;
      for (int id = i + 1; id <= n; id++) {
        if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
          swapid = id; break;
        }
      }
      if (swapid == -1)
        continue;
      else {
        for (int j = 1; j <= n + 1; j++)
          swap(co[i][j], co[swapid][j]);
      }
    }
    for (int id = i + 1; id <= n; id++) {
      if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
        frac mul = co[id][i] / co[i][i];
        for (int j = 1; j <= n + 1; j++)
          co[id][j] = co[id][j] - co[i][j] * mul;
      }
    }
  }

  for (int i = n; i >= 1; i--) {
    if (! (co[i][i].a == 0 && co[i][i].b == 1)) {
      co[i][n + 1] = co[i][n + 1] / co[i][i];
      co[i][i] = {1, 1};
    } else
      continue;
    for (int id = 1; id < i; id++) {
      if (! (co[id][i].a == 0 && co[id][i].b == 1)) {
        frac mul = co[id][i] / co[i][i];
        for (int j = 1; j <= n + 1; j++)
          co[id][j] = co[id][j] - co[i][j] * mul;
      }
    }
  }
  bool check_inf = false, check_no = false;
  for (int i = 1; i <= n; i++) {
    bool allzro = true;
    for (int j = 1; j <= n; j++)
      if (! (co[i][j].a == 0 && co[i][j].b == 1))
        allzro = false;
    if (allzro) {
      if (! (co[i][n + 1].a == 0 && co[i][n + 1].b == 1))
        check_no = true;
      else
        check_inf = true;
    }
  }
  if (check_no) {
    cout << "no solutions\n";
  } else if (check_inf) {
    cout << "infinite many solutions\n";
  } else {
    for (int i = 1; i <= n; i++)
      cout << "X" << i << " = " << co[i][n + 1] << '\n';
  }
}
```

6.13 josephus

```
// 有 n 個人，第偶數個報數的人被刪掉，問第 k 個被踢掉的是誰
int solve(int n, int k) {
  if (n == 1) return 1;
  if (k <= (n + 1) / 2) {
    if (2 * k > n)
      return 2 * k % n;
    else
      return 2 * k;
  } else {
    int res = solve(n / 2, k - (n + 1) / 2);
    if (n & 1)
      return 2 * res + 1;
    else
      return 2 * res - 1;
  }
}
```

6.14 lucas

```
// 對於很大的 C^n_{m} 對質數 p 取模，只要 p 不大就可以用
int Lucas(int n, int m, int p) {
  if (m == 0) return 1;
  return (C(n % p, m % p, p) * Lucas(n / p, m / p, p) % p);
}
```

6.15 math block

```
// the sum of floor(n / i) for i = 1 ~ n
int f(int n) {
  int ans = 0;
  int l = 1, r = 0;
  while (l <= n) {
    r = n / (n / l);
    ans += (r - l + 1) * (n / l);
    l = r + 1;
  }
  return ans;
}
// the sum of ceil(n / i) for i = 1 ~ n
int f(int n) {
  int ans = 0;
  int l = 1, r = 0;
  while (l < n) {
    r = (n - 1) / ((n - 1) / l);
    ans += (r - l + 1) * ((n + l - 1) / l);
    l = r + 1;
  }
```

```
  if (l == n) ans += (n + l - 1) / l;
  return ans;
}
```

6.16 miller rabin

```
struct Miller_Rabin {
  static constexpr uint32_t ws32[3] = {2, 7, 61};
  static constexpr uint64_t ws64[7] = {2, 325, 9375, 28178,
       450775, 9780504, 1795265022};
  template <class uint>
  static constexpr uint fast_pow(uint b, uint p, uint mod) {
    using Uint = conditional_t<is_same_v<uint, uint32_t>,
         uint64_t, __uint128_t>;
    uint ret{1};
    for (; p; p >>= 1) {
      if (p & 1) ret = Uint{ret} * b % mod;
      b = Uint{b} * b % mod;
    }
    return ret;
  }
  template <class uint, class = enable_if_t<is_unsigned_v<uint
       >>>
  static constexpr bool is_prime(uint n) {
    const auto& witness = []() -> const auto& {
      if constexpr (is_same_v<uint, uint32_t>) return ws32;
      else return ws64;
    }();
    using Uint = conditional_t<is_same_v<uint, uint32_t>,
         uint64_t, __uint128_t>;
    if (n < 3 || !(n & 1)) return n == 2;
    auto u = n - 1, t = (uint)__builtin_ctzll(u);
    u >>= t;
    for (auto x : witness) {
      auto a = x % n;
      if (a == 0 || a == 1 || a == n - 1) continue;
      auto v = fast_pow(a, u, n);
      uint i = 0;
      for (; v != 1 && v != n - 1 && i < t; i++) v = Uint{v} *
           v % n;
      if (v != n - 1 && i > 0) return false;
    }
    return true;
  }
  template <class I, class = enable_if_t<is_integral_v<I>>>
  constexpr bool operator()(I x) const {
    if constexpr (is_signed_v<I>) if (x < 0) return false;
    using U = make_unsigned_t<I>;
    return is_prime((U)x);
  }
} is_prime;
```

6.17 phi

```
// phi(n) = 小於 n 並與 n 互質的正整數數量。
// O(sqrt(n))，回傳 phi(n)
int phi(int n) {
  int ret = n;
  for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      while (n % i == 0) n /= i;
      ret = ret * (i - 1) / i;
    }
  }
  if (n > 1) ret = ret * (n - 1) / n;
  return ret;
}
// O(n log n)，回傳 1 ~ n 的 phi 值
vector<int> phi_1_to_n(int n) {
  vector<int> phi(n + 1);
  phi[0] = 0; phi[1] = 1;
  for (int i = 2; i <= n; i++) phi[i] = i - 1;
  for (int i = 2; i <= n; i++)
    for (int j = 2 * i; j <= n; j += i) // 枚舉所有倍數
      phi[j] -= phi[i];
  return phi;
}
```

6.18 phi lemma

若 a, m 互質，則：

$$a^n \bmod m = a^{n \bmod \varphi(m)} \bmod m$$

若 a, m 可能是任何數，則：

$$a^{\varphi(m)+[n \bmod \varphi(m)]} \bmod m$$

6.19 primes er

```
// Time Complexity : O(nloglogn)
const int N = 200005;
bool is_prime[N];
vector <int> primes;
void build() {
  fill(is_prime, is_prime + N, true);
  is_prime[0] = is_prime[1] = false;
  for (int i = 2; i <= 200000; i++) {
    if (is_prime[i]) {
      primes.push_back(i);
      if (1ll * i * i <= 200000)
        for (int j = i * i; j <= 200000; j += i)
          is_prime[j] = false;
    }
  }
}
```

6.20 primes linear

```
// Time Complexity : O(n)
// if lpf(i) = i, then it means that i is a prime.
// else lpf(i) is the smallest prime factor.
// The 199999-th of the prime is 2750131
bool is_prime[2750135];
int lpf[2750135];
vector<int> primes;
void init() {
  fill(is_prime, is_prime + 2750135, true);
  for (int i = 2; i <= 2750131; i++) {
    if (is_prime[i]) {
      primes.push_back(i);
      lpf[i] = i;
    }
    for (int p : primes) {
      if (p * i > 2750131) break;
      is_prime[p * i] = false;
      lpf[p * i] = p;
      if (i % p == 0) break;
    }
  }
}
```

6.21 two equations

$$\begin{cases} ax + by = c \\ dx + ey = f \end{cases} = \begin{cases} x = \frac{ed-bf}{ad-bc} \\ y = \frac{af-ec}{ad-bc} \end{cases}$$

若 $x = \frac{0}{0}$ 且 $y = \frac{0}{0}$，則代表無限多組解。若 $x = \frac{*}{0}$ 且 $y = \frac{*}{0}$，則代表無解。

7 other

7.1 aliens dp

```
const int N = 2000005;
int n, k;
int a[N];
pii dp0[N], dp1[N];
pii calc(int c) {
  dp0[1] = {0, 0}; dp1[1] = {c - a[1], 1};
  for (int i = 2; i <= n; i++) {
    dp0[i] = max(dp0[i - 1], dp1[i - 1] + make_pair(+a[i], 0));
    dp1[i] = max(dp1[i - 1], dp0[i - 1] + make_pair(-a[i] + c,
         1));
  }
  return dp0[n];
}
void solve() {
  pii p = calc(0);
  if (p.S <= k) {
    cout << p.F << "\n";
    return;
  }
  int l = -1e5, r = 1e6; pii res;
  while (l + 1 < r) {
    int mid = (l + r) >> 1;
    res = calc(mid);
    if (res.S == k) {
      cout << res.F - mid * k << "\n";
      return;
    } else if (res.S > k)
      r = mid;
    else
      l = mid;
```

```
  }
  pii p1 = calc(l), p2 = calc(r);
  int dp_1 = p1.F - l * p1.S;
  int dp_2 = p2.F - r * p2.S;
  cout << (dp_1 + (dp_2 - dp_1) * (k - p1.S) / (p2.S - p1.S))
       << "\n";
}
```

7.2 all LCS

```
void all_lcs(string s, string t) { // 0-base
    vector<int> h(sz(t));
    iota(all(h), 0);
    for (int a = 0; a < sz(s); ++a) {
        int v = -1;
        for (int c = 0; c < sz(t); ++c)
            if (s[a] == t[c] || h[c] < v)
                swap(h[c], v);
        // LCS(s[0, a], t[b, c]) =
        // c - b + 1 - sum([h[i] >= b] | i <= c)
        // h[i] might become -1 !!
    }
}
```

7.3 dc dp

```
const int N = 3005;
int n, k, a[N], pre[N], dp[N][N];
int cost(int l, int r) {}
void dc(int j, int l, int r, int best_l, int best_r) {
  if (l > r) return;
  int mid = (l + r) >> 1, pos = 1;
  for (int i = best_l; i <= best_r; i++) {
    if (dp[i][j - 1] + cost(i + 1, mid) < dp[mid][j]) {
        pos = i;
        dp[mid][j] = dp[i][j - 1] + cost(i + 1, mid);
    }
  }
  dc(j, l, mid - 1, best_l, pos);
  dc(j, mid + 1, r, pos, best_r);
}
void solve() {
  for (int i = 1; i <= n; i++)
    dp[i][1] = pre[i] * pre[i];
  for (int j = 2; j <= k; j++)
    dc(j, j, n, j - 1, n);
}
```

7.4 digit dp

```
// 原題：計算 [l, r] 中兩兩相鄰數字不同的數字數量
// 注意邊界
int dp[20][10]; // i 位數，尾數是 j
void init() {
  for (int j = 0; j <= 9; j++) dp[1][j] = 1;
  for (int i = 2; i <= 18; i++)
    for (int now = 0; now <= 9; now++) // 這個位數的數字
      for (int pre = 0; pre <= 9; pre++) // 上個位數的數字
        if (now != pre)
          dp[i][now] += dp[i - 1][pre];
}
int calc(int x) { // 小於等於 x 的數量
  if (x == 0) return 0;
  int arr[20] = {}, len = 0;
  while (x > 0) arr[++len] = x % 10, x /= 10;
  // 計算 len 位數 的數量
  int cnt = 0, pre = -1;
  for (int idx = len; idx >= 1; idx--) {
    for (int now = (idx == len); now < arr[idx]; now++)
      if (now != pre)
        cnt += dp[idx][now];
    if (arr[idx] == pre) break;
    pre = arr[idx];
    if (idx == 1) cnt++; // 特判
  }
  // 計算 小於 len 位數 的數量
  for (int i = 1; i < len; i++)
    for (int j = 1; j <= 9; j++)
      cnt += dp[i][j];
  return cnt;
}
```

7.5 geometry

```
double eps = 0.000001;
pdd operator + (pdd a, pdd b) { return {a.X + b.X, a.Y + b.Y}; }
pdd operator - (pdd a, pdd b) { return {a.X - b.X, a.Y - b.Y}; }
double dot(pdd a, pdd b) { return a.X * b.X + a.Y * b.Y; }
double cross(pdd a, pdd b) { return a.X * b.Y - a.Y * b.X; }
int sign(double x) { return (fabs(x) < eps ? 0 : (x > 0 ? 1 : -1)); }
int ori(pdd a, pdd b, pdd c) { return sign(cross(b - a, c - a)); }
// c is between a and b
bool btw(pdd a, pdd b, pdd c) {
  return (sign(cross(a - c, b - c)) == 0 && sign(dot(a - c, b - c)) <= 0);
}
// ab is touch cd
bool banana(pdd a, pdd b, pdd c, pdd d) {
  int a123 = ori(a, b, c), a124 = ori(a, b, d);
  int a341 = ori(c, d, a), a342 = ori(c, d, b);
  if (btw(a, b, c) || btw(a, b, d) || btw(c, d, a) || btw(c, d, b))
    return true;
  return (a123 * a124 < 0 && a341 * a342 < 0);
}
vector <pdd> convex_hull(vector <pdd> pts) {
  sort(pts.begin(), pts.end());
  pts.resize(unique(pts.begin(), pts.end()) - pts.begin());
  vector <pdd> hull(1, pts[0]);
  for (int t = 0; t < 2; t++) {
    int sz = hull.size();
    for (int i = 1; i < (int)pts.size(); i++) {
      while ((int)hull.size() > sz && ori(hull[(int)hull.size() - 2], hull.back(), pts[i]) < 0)
        hull.pop_back();
      hull.push_back(pts[i]);
    }
    reverse(pts.begin(), pts.end());
  }
  hull.pop_back();
  return hull;
}
double area(vector <pdd> pt) {
  double cnt = 0.0;
  for (int i = 0; i < (int)pt.size() - 1; i++)
    cnt += cross(pt[i], pt[i + 1]);
  cnt += cross(pt[(int)pt.size() - 1], pt[0]);
  return fabs(cnt);
}
void solve() {
  int n;
  cin >> n;
  vector <pdd> pts;
  pts.resize(n);
  for (int i = 0; i < n; i++)
    cin >> pts[i].X >> pts[i].Y;
  int ans = area(convex_hull(pts));
  if (ans % 2 == 1)
    cout << ans / 2 << ".5\n";
  else
    cout << ans / 2 << ".0\n";
}
```

7.6 line container

```
// only works for integer coordinates!! maintain max
struct Line {
  mutable ll a, b, p;
  bool operator<(const Line &rhs) const { return a < rhs.a; }
  bool operator<(ll x) const { return p < x; }
};
struct DynamicHull : multiset<Line, less<>> {
  static const ll kInf = 1e18;
  ll Div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) { x->p = kInf; return 0; }
    if (x->a == y->a) x->p = x->b > y->b ? kInf : -kInf;
    else x->p = Div(y->b - x->b, x->a - y->a);
    return x->p >= y->p;
  }
  void addline(ll a, ll b) { // ax + b
    auto z = insert({a, b, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p) isect(x, erase(y));
  }
  ll query(ll x) {
    auto l = *lower_bound(x);
    return l.a * x + l.b;
  }
};
```

```
};
```

7.7 mo

```
struct MoSolver {
  struct query {
    int l, r, id;
    bool operator < (const query &o) {
      if (l / C == o.l / C) return (l / C) & 1 ? r > o.r : r < o.r;
      return l / C < o.l / C;
    }
  };
  int cur_ans;
  vector <int> ans;
  void add(int x) {
    // do something
  }
  void sub(int x) {
    // do something
  }
  vector <query> Q;
  void add_query(int l, int r, int id) {
    // [l, r)
    Q.push_back({l, r, id});
    ans.push_back(0);
  }
  void run() {
    sort(Q.begin(), Q.end());
    int pl = 0, pr = 0;
    cur_ans = 0;
    for (query &i : Q) {
      while (pl > i.l)
        add(a[--pl]);
      while (pr < i.r)
        add(a[pr++]);
      while (pl < i.l)
        sub(a[pl++]);
      while (pr > i.r)
        sub(a[--pr]);
      ans[i.id] = cur;
    }
  }
};
```

7.8 number split

```
// dp[i][x] = 要將整數 x 拆成 i 堆的「組合數」
dp[i + 1][x + 1] += dp[i][x]; // 創造新的一堆
dp[i][x + i] += dp[i][x];     // 把每一堆都增加 1
```

7.9 python

```
from [decimal, fractions, math, random] import *
arr = list(map(int, input().split())) # input
setcontext(Context(prec=10, Emax=MAX_EMAX, rounding=ROUND_FLOOR))
Decimal('1.1') / Decimal('0.2')
Fraction(3, 7)
Fraction(Decimal('1.14'))
Fraction('1.2').limit_denominator(4).numerator
Fraction(cos(pi / 3)).limit_denominator()
S = set(), S.add((a, b)), S.remove((a, b)) # set
if not (a, b) in S:
D = dict(), D[(a, b)] = 1, del D[(a, b)] # dict
for (a, b) in D.items():
arr = [randint(1, C) for i in range(N)]
choice([8, 6, 4, 1]) # random pick one
```

7.10 sos dp

```
// 時間複雜度為 O(n * 2 ^ n)
// 計算 dp[i] = i 所有 bit mask 子集的和
for (int i = 0; i < n; i++)
  for (int mask = 0; mask < (1 << n); mask++)
    if ((mask >> i) & 1)
      dp[mask] += dp[mask ^ (1 << i)];
```

7.11 subset

```
// 枚舉子集，時間複雜度 O(3 ^ n)
for (int i = 0 ; i < (1 << n); i++) {
  for (int m = i; m; m = (m - 1) & i) {
    // m 是 i 的 非 空 子 集
  }
}
```

7.12 xor basis

```
// 給一數字集合 S，求能 XOR 出多少數字
// 答案等於 2 ^ {basis 的大小}
vector<int> basis;
void add_vector(int x) {
  for (int v : basis) x = min(x, x ^ v);
  if (x) basis.push_back(x);
}
// 給一數字集合 S，求能不能 XOR 出 x
bool check(int x) {
  for (int v : basis) x = min(x, x ^ v);
  return x;
}
// 給一數字集合 S，求 XOR 出最大的數字
int get_max() {
  int ans = 0;
  for (auto v : basis) ans = max(ans, ans ^ v);
  return ans;
}
```

8 polynomial

8.1 FFT

```
using T = complex<double>;
const double PI = acos(-1);
struct FFT {
  T w[N];
  FFT() {
    for (int i = 0; i < N; ++i) w[i] = T(cos(2 * PI * i / N), sin(2 * PI * i / N));
  }
  void operator()(vector<T>& a, bool inv = false) {
    int n = sz(a);
    for (int i = 1, j = 0; i < n; ++i) {
      for (int k = n >> 1; (j ^= k) < k; k >>= 1);
      if (i < j) swap(a[i], a[j]);
    }
    for (int L = 1; L < n; L <<= 1)
      for (int i = 0, st = N / (2 * L); i < n; i += 2 * L)
        for (int j = 0; j < L; ++j) {
          T x = a[i + j + L] * w[j * st];
          a[i + j + L] = a[i + j] - x;
          a[i + j] += x;
        }
    if (inv) {
      reverse(1 + all(a));
      T invn = 1.0 / n;
      for (int i = 0; i < n; ++i) a[i] = a[i] * invn;
    }
  }
} ntt;
// after mul, round i.real()
// for (int i = 0; i < m; ++i) a[i] = llround(a[i].real());
```

8.2 NTT

```
// mul, add, sub, Pow
struct NTT {
  int w[N];
  NTT() {
    int dw = Pow(G, (mod - 1) / N);
    w[0] = 1;
    for (int i = 1; i < N; ++i)
      w[i] = mul(w[i - 1], dw);
  } // 0 <= a[i] < P
  void operator()(vector<int>& a, bool inv = false) {
    int n = sz(a);
    for (int j = 1, x = 0; j < n - 1; ++j) {
      for (int k = n >> 1; (x ^= k) < k; k >>= 1);
      if (j < x) swap(a[x], a[j]);
    }
    for (int L = 2; L <= n; L <<= 1) {
      int dx = N / L, dl = L >> 1;
```

```
      for (int i = 0; i < n; i += L) {
        for (int j = i, x = 0; j < i + dl; ++j, x += dx) {
          int tmp = mul(a[j + dl], w[x]);
          a[j + dl] = sub(a[j], tmp);
          a[j] = add(a[j], tmp);
        }
      }
    }
    if (inv) {
      reverse(1 + all(a));
      int invn = Pow(n, mod - 2);
      for (int i = 0; i < n; ++i)
        a[i] = mul(a[i], invn);
    }
  }
} ntt;
```

8.3 NTT prime

Prime	Root	Prime	Root
7681	17	167772161	3
12289	11	104857601	3
40961	3	985661441	3
65537	3	998244353	3
786433	10	1107296257	10
5767169	3	2013265921	31
7340033	3	2810183681	11
23068673	3	2885681153	3
469762049	3	605028353	3
2061584302081	7	1945555039024054273	5
2748779069441	3	9223372036737335297	3

8.4 operation

```
typedef vector<int> Poly;
Poly Mul(Poly a, Poly b, int bound = N) {
  int m = sz(a) + sz(b) - 1, n = 1;
  while (n < m) n <<= 1;
  a.resize(n), b.resize(n);
  ntt(a), ntt(b);
  for (int i = 0; i < n; ++i) a[i] = mul(a[i], b[i]);
  ntt(a, true), a.resize(min(m, bound));
  return a;
}
Poly Inverse(Poly a) {
  // O(NlogN), a[0] != 0
  int n = sz(a);
  Poly res(1, Pow(a[0], mod - 2));
  for (int m = 1; m < n; m <<= 1) {
    if (n < m * 2) a.resize(m * 2);
    Poly v1(a.begin(), a.begin() + m * 2), v2 = res;
    v1.resize(m * 4), v2.resize(m * 4);
    ntt(v1), ntt(v2);
    for (int i = 0; i < m * 4; ++i)
      v1[i] = mul(mul(v1[i], v2[i]), v2[i]);
    ntt(v1, true);
    res.resize(m * 2);
    for (int i = 0; i < m; ++i)
      res[i] = add(res[i], res[i]);
    for (int i = 0; i < m * 2; ++i)
      res[i] = sub(res[i], v1[i]);
  }
  res.resize(n);
  return res;
}
pair <Poly, Poly> Divide(Poly a, Poly b) {
  // a = bQ + R, O(NlogN), b.back() != 0
  int n = sz(a), m = sz(b), k = n - m + 1;
  if (n < m) return {{0}, a};
  Poly ra = a, rb = b;
  reverse(all(ra)), ra.resize(k);
  reverse(all(rb)), rb.resize(k);
  Poly Q = Mul(ra, Inverse(rb), k);
  reverse(all(Q));
  Poly res = Mul(b, Q), R(m - 1);
  for (int i = 0; i < m - 1; ++i)
    R[i] = sub(a[i], res[i]);
  return {Q, R};
}
Poly SqrtImpl(Poly a) {
  if (a.empty()) return {0};
  int z = QuadraticResidue(a[0], mod), n = sz(a);
  if (z == -1) return {-1};
  Poly q(1, z);
  const int inv2 = (mod + 1) / 2;
  for (int m = 1; m < n; m <<= 1) {
    if (n < m * 2) a.resize(m * 2);
    q.resize(m * 2);
    Poly f2 = Mul(q, q, m * 2);
    for (int i = 0; i < m * 2; ++i)
      f2[i] = sub(f2[i], a[i]);
    f2 = Mul(f2, Inverse(q), m * 2);
    for (int i = 0; i < m * 2; ++i)
      q[i] = sub(q[i], mul(f2[i], inv2));
  }
  q.resize(n);
  return q;
}
Poly Sqrt(Poly a) {
  // O(NlogN), return {-1} if not exists
  int n = sz(a), m = 0;
  while (m < n && a[m] == 0) m++;
  if (m == n) return Poly(n);
  if (m & 1) return {-1};
  Poly s = SqrtImpl(Poly(a.begin() + m, a.end()));
  if (s[0] == -1) return {-1};
  Poly res(n);
  for (int i = 0; i < sz(s); ++i)
    res[i + m / 2] = s[i];
  return res;
}
Poly Derivative(Poly a) {
  int n = sz(a);
  Poly res(n - 1);
  for (int i = 0; i < n - 1; ++i)
    res[i] = mul(a[i + 1], i + 1);
  return res;
}
Poly Integral(Poly a) {
  int n = sz(a);
  Poly res(n + 1);
  for (int i = 0; i < n; ++i)
    res[i + 1] = mul(a[i], Pow(i + 1, mod - 2));
  return res;
}
Poly Ln(Poly a) {
  // O(NlogN), a[0] = 1
  int n = sz(a);
  if (n == 1) return {0};
  Poly d = Derivative(a);
  a.pop_back();
  return Integral(Mul(d, Inverse(a), n - 1));
}
Poly Exp(Poly a) {
  // O(NlogN), a[0] = 0
  int n = sz(a);
  Poly q(1, 1);
  a[0] = add(a[0], 1);
  for (int m = 1; m < n; m <<= 1) {
    if (n < m * 2) a.resize(m * 2);
    Poly g(a.begin(), a.begin() + m * 2), h(all(q));
    h.resize(m * 2), h = Ln(h);
    for (int i = 0; i < m * 2; ++i)
      g[i] = sub(g[i], h[i]);
    q = Mul(g, q, m * 2);
  }
  q.resize(n);
  return q;
}
Poly PolyPow(Poly a, ll k) {
  int n = sz(a), m = 0;
  Poly ans(n, 0);
  while (m < n && a[m] == 0) m++;
  if (k && m && (k >= n || k * m >= n)) return ans;
  if (m == n) return ans[0] = 1, ans;
  int lead = m * k;
  Poly b(a.begin() + m, a.end());
  int base = Pow(b[0], k), inv = Pow(b[0], mod - 2);
  for (int i = 0; i < n - m; ++i)
    b[i] = mul(b[i], inv);
  b = Ln(b);
  for (int i = 0; i < n - m; ++i)
    b[i] = mul(b[i], k % mod);
  b = Exp(b);
  for (int i = lead; i < n; ++i)
    ans[i] = mul(b[i - lead], base);
  return ans;
}
vector <int> Evaluate(Poly a, vector <int> x) {
  if (x.empty()) return {};
  int n = sz(x);
  vector <Poly> up(n * 2);
  for (int i = 0; i < n; ++i)
    up[i + n] = {sub(0, x[i]), 1};
  for (int i = n - 1; i > 0; --i)
    up[i] = Mul(up[i * 2], up[i * 2 + 1]);
  vector <Poly> down(n * 2);
  down[1] = Divide(a, up[1]).second;
  for (int i = 2; i < n * 2; ++i)
    down[i] = Divide(down[i >> 1], up[i]).second;
  Poly y(n);
  for (int i = 0; i < n; ++i) y[i] = down[i + n][0];
  return y;
}
Poly Interpolate(vector <int> x, vector <int> y) {
  int n = sz(x);
  vector <Poly> up(n * 2);
  for (int i = 0; i < n; ++i)
    up[i + n] = {sub(0, x[i]), 1};
```

```
  for (int i = n - 1; i > 0; --i)
    up[i] = Mul(up[i * 2], up[i * 2 + 1]);
  Poly a = Evaluate(Derivative(up[1]), x);
  for (int i = 0; i < n; ++i)
    a[i] = mul(y[i], Pow(a[i], mod - 2));
  vector <Poly> down(n * 2);
  for (int i = 0; i < n; ++i) down[i + n] = {a[i]};
  for (int i = n - 1; i > 0; --i) {
    Poly lhs = Mul(down[i * 2], up[i * 2 + 1]);
    Poly rhs = Mul(down[i * 2 + 1], up[i * 2]);
    down[i].resize(sz(lhs));
    for (int j = 0; j < sz(lhs); ++j)
      down[i][j] = add(lhs[j], rhs[j]);
  }
  return down[1];
}
Poly TaylorShift(Poly a, int c) {
  // return sum a_i(x + c)^i;
  // fac[i] = i!, facp[i] = inv(i!)
  int n = sz(a);
  for (int i = 0; i < n; ++i) a[i] = mul(a[i], fac[i]);
  reverse(all(a));
  Poly b(n);
  int w = 1;
  for (int i = 0; i < n; ++i)
    b[i] = mul(facp[i], w), w = mul(w, c);
  a = Mul(a, b, n), reverse(all(a));
  for (int i = 0; i < n; ++i) a[i] = mul(a[i],facp[i]);
  return a;
}
vector<int> SamplingShift(vector<int> a, int c, int m){
  // given f(0), f(1), ..., f(n - 1)
  // return f(c), f(c + 1), ..., f(c + m - 1)
  int n = sz(a); // 4d649d
  for (int i = 0; i < n; ++i) a[i] = mul(a[i],facp[i]);
  Poly b(n);
  for (int i = 0; i < n; ++i) {
    b[i] = facp[i];
    if (i & 1) b[i] = sub(0, b[i]);
  }
  a = Mul(a, b, n);
  for (int i = 0; i < n; ++i) a[i] = mul(a[i], fac[i]);
  reverse(all(a));
  int w = 1;
  for (int i = 0; i < n; ++i)
    b[i] = mul(facp[i], w), w = mul(w, sub(c, i));
  a = Mul(a, b, n);
  reverse(all(a));
  for (int i = 0; i < n; ++i) a[i] = mul(a[i],facp[i]);
  a.resize(m), b.resize(m);
  for (int i = 0; i < m; ++i) b[i] = facp[i];
  a = Mul(a, b, m);
  for (int i = 0; i < m; ++i) a[i] = mul(a[i], fac[i]);
  return a;
}
Poly power_proj(vector<int> w, Poly f, int m) {
  // return sum_j w_j[x^j]f^i for i=0,1,...,m
  int n = 1;
  while (n < sz(f)) n <<= 1;
  f.resize(n), w.resize(n), reverse(all(w));
  int k = 1, n2 = 2 * n, n4 = 4 * n;
  Poly _P(n2), _Q(n2);
  for (int i = 0; i < n; ++i)
    _P[i] = w[i], _Q[i] = sub(0, f[i]);
  while (n > 1) {
    Poly R(n2);
    for (int i = 0; i < n2; ++i)
      R[i] = (i & 1 ? sub(0, _Q[i]) : _Q[i]);
    Poly PQ = Mul(_P, R), QQ = Mul(_Q, R);
    PQ.resize(n4), QQ.resize(n4);
    for (int i = 0, j = n2; i < n2; ++i, ++j) {
      PQ[j] = add(PQ[j], _P[i]);
      QQ[j] = add(QQ[j], add(_Q[i], R[i]));
    }
    fill(all(_P), 0), fill(all(_Q), 0);
    for (int j = 0; j < 2 * k; ++j)
      for (int i = 0; i < n / 2; ++i) {
        _P[n * j + i] = PQ[2 * n * j + 2 * i + 1];
        _Q[n * j + i] = QQ[2 * n * j + 2 * i + 0];
      }
    n /= 2, k *= 2;
  }
  Poly p(k);
  for (int i = 0; i < k; ++i) p[i] = _P[2 * i];
  reverse(all(p)), p.resize(m + 1);
  return p;
}
```

9 string

9.1 hash

```
const int p = 1e6 + 3;
const int mod = 1e9 + 7;
int hash(string &s) {
  int res = 0;
  for (int i = 0; i < s.size(); i++) {
    res *= p;
    res += s[i] - 'a' + 1;
    res %= mod;
  }
  return res;
}
int pow_p[1000005];
vector <int> rh;
void build(string &s) {
  rh.resize(s.size() + 1);
  pow_p[0] = 1;
  for (int i = 1; i <= s.size(); i++)
    pow_p[i] = (pow_p[i - 1] * p) % mod;
  rh[0] = s[0] - 'a' + 1;
  for (int i = 1; i < s.size(); i++) {
    rh[i] = rh[i - 1] * p + (s[i] - 'a' + 1);
    rh[i] %= mod;
  }
}
int query(int l, int r) {
  int res = rh[r] - (l > 0 ? rh[l - 1] * pow_p[r - l + 1] : 0);
  res = (res % mod + mod) % mod;
  return res;
}
```

9.2 kmp

```
vector <int> F;
void build_failure_function(string &s) {
  F.clear();
  F.resize(s.size(), -1);
  for (int i = 1; i < s.size(); i++) {
    int j = F[i - 1];
    while (j != -1 && s[i] != s[j + 1])
      j = F[j];
    if (s[i] == s[j + 1])
      F[i] = j + 1;
  }
}
void KMP_matching(string &a, string &b) {
  for (int i = 0, j = -1; i < a.size(); i++) {
    while (j != -1 && a[i] != b[j + 1])
      j = F[j];
    if (a[i] == b[j + 1])
      j++;
    if (j + 1 == b.size()) {
      cout << "found a matching start at " << i - j << "\n";
      j = F[j];
    }
  }
}
```

9.3 manacher

```
string s;
int extend(int l, int r, int N) {
  int i = 0;
  while (l - i >= 0 && r + i < N && s[l - i] == s[r + i])
    i++;
  return i;
}
int Longest_Palindromic_Substring(string &t) {
  int N = t.length();
  s.resize(2 * N + 1, '$');
  for (int i = 0; i < N; i++)
    s[2 * i + 1] = t[i];
  N = 2 * N + 1;
  vector <int> res;
  res.resize(N, 0);
  res[0] = 1;
  for (int i = 1, mid = 0, R = 0; i < N; i++) {
    int j = mid - (i - mid);
    int lst = R + 1 - i;
    if (i > R) {
      res[i] = extend(i, i, N);
      mid = i;
      R = i + res[i] - 1;
    } else if (res[j] == lst) {
      res[i] = lst + extend(i - R, i + R, N);
      mid = i;
      R = i + res[i] - 1;
    } else {
      res[i] = min(res[j], lst);
    }
  }
  int mx = -1, idx = -1;
```

```
  for (int i = 0; i < N; i++) {
    if (res[i] > mx) {
      mx = res[i];
      idx = i;
    }
  }
  cout << "Longest Palindromic Substring Length is " << (mx + 1) / 2 << "\n";
  for (int i = idx - res[idx] + 1; i <= idx + res[idx] + 1; i ++)
    if (i & 1)
      cout << s[i];
  cout << "\n";
}
```

9.4 min rotation

```
int minRotation(const vector<int>& a) {
    int n = a.size(), i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        int x = a[(i + k) % n], y = a[(j + k) % n];
        if (x == y) { k++; continue; }
        if (x > y) i += k + 1;
        else       j += k + 1;
        if (i == j) j++;
        k = 0;
    }
    return min(i, j);
}
```

9.5 string trie

```
struct node {
  node* link[26] = {nullptr};
  bool flag = false;
  bool contain_key(char ch) { return (link[ch - 'a'] != nullptr); }
  void put(char ch, node* _node) { link[ch - 'a'] = _node; }
  node* get(char ch) { return link[ch - 'a']; }
  void set_end() { flag = true; }
  bool is_end() { return flag; }
};
struct Trie {
  node* root;
  Trie() { root = new node(); }
  void insert(string s) { // insert a string
    node* now = root;
    for (int i = 0; i < (int)s.size(); i++) {
      if (!now->contain_key(s[i])) now->put(s[i], new node());
      now = now->get(s[i]);
    }
    now->set_end();
  }
    // return if there is a word of the given string
  bool query(string s) {
    node* now = root;
    for (int i = 0; i < (int)s.size(); i++) {
      if (!now->contain_key(s[i])) return false;
      now = now->get(s[i]);
    }
    return now->is_end();
  }
  // return if there is any word that start with the given prefix
  bool query_prefix(string s) {
    node* now = root;
    for (int i = 0; i < (int)s.size(); i++) {
      if (!now->contain_key(s[i])) return false;
      now = now->get(s[i]);
    }
    return true;
  }
};
```

9.6 suffix array

```
// s  = "banana"
// sa = [5 3 1 0 4 2]
// rk = [3 2 5 1 4 0]
const int N = 1e6 + 5;
int sa[N], tmp[2][N], c[N], rk[N], lcp[N];
vector<vector<int>> st;
void buildSA(const string &s) {
  int *x = tmp[0], *y = tmp[1], m = 256, n = s.size();
  for (int i = 0; i < m; ++i) c[i] = 0;
  for (int i = 0; i < n; ++i) c[x[i] = (unsigned char)s[i]]++;
  for (int i = 1; i < m; ++i) c[i] += c[i - 1];
  for (int i = n - 1; ~i; --i) sa[--c[x[i]]] = i;
  for (int k = 1; k < n; k <<= 1) {
    for (int i = 0; i < m; ++i) c[i] = 0;
    for (int i = 0; i < n; ++i) c[x[i]]++;
    for (int i = 1; i < m; ++i) c[i] += c[i - 1];
    int p = 0;
    for (int i = n - k; i < n; ++i) y[p++] = i;
    for (int i = 0; i < n; ++i) if (sa[i] >= k)
      y[p++] = sa[i] - k;
    for (int i = n - 1; ~i; --i)
      sa[--c[x[y[i]]]] = y[i];
    y[sa[0]] = p = 0;
    for (int i = 1; i < n; ++i) {
      int a = sa[i], b = sa[i - 1];
      if (!(x[a] == x[b] && a + k < n && b + k < n && x[a + k] == x[b + k])) p++;
      y[sa[i]] = p;
    }
    if (n == p + 1) break;
    swap(x, y), m = p + 1;
  }
}
void buildLCP(const string &s) {
  // lcp[i] = LCP(sa[i - 1], sa[i])
  // lcp(i, j) = query_lcp_min [rk[i] + 1, rk[j] + 1)
  int n = s.length(), val = 0;
  for (int i = 0; i < n; ++i) rk[sa[i]] = i;
  for (int i = 0; i < n; ++i) {
    if (!rk[i]) lcp[rk[i]] = 0;
    else {
      if (val) val--;
      int p = sa[rk[i] - 1];
      while (val + i < n && val + p < n && s[val + i] == s[val + p]) val++;
      lcp[rk[i]] = val;
    }
  }
}
void buildST(int n) {
  // st[j][i] = min(lcp[i], ..., lcp[i + 2^j - 1])
  st.assign(__lg(n) + 1, vector<int>(n));
  for (int i = 0; i < n; ++i) st[0][i] = lcp[i];
  for (int j = 1; (1 << j) <= n; ++j)
    for (int i = 0; i + (1 << j) <= n; ++i)
      st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
}
int queryMin(int l, int r) {
  // min of lcp[l, r)
  int j = __lg(r - l);
  return min(st[j][l], st[j][r - (1 << j)]);
}
int get_lcp(int l1, int r1, int l2, int r2) {
  // LCP(s[l1, r1), s[l2, r2))
  int len = min(r1 - l1, r2 - l2);
  if (len <= 0) return 0;
  if (l1 == l2) return len;
  int a = rk[l1], b = rk[l2];
  if (a > b) swap(a, b);
  return min(len, queryMin(a + 1, b + 1));
}
```

9.7 xor trie

```
struct node {
  node* link[2];
  bool contain_key(int val) { return (link[val] != nullptr); }
  node* get(int val) { return link[val]; }
  void put(int val, node* _node) { link[val] = _node; }
};
struct Trie {
  node* root;
  Trie() { root = new node(); }
  void update(int x) {
    node* now = root;
    for (int i = 30; i >= 0; i--) {
      if (x & (1 << i)) {
        if (!now->contain_key(1)) now->put(1, new node());
        now = now->get(1);
      } else {
        if (!now->contain_key(0)) now->put(0, new node());
        now = now->get(0);
      }
    }
  }
  int query(int x) { // query max xor with number x
    node* now = root;
    int mx = 0;
    for (int i = 30; i >= 0; i--) {
      int nowbit = (x >> i) & 1;
      int target = nowbit ^ 1;
      if (now->contain_key(target)) {
        mx += (1 << i);
        now = now->get(target);
```

```
      } else {
        now = now->get(nowbit);
      }
    }
    return mx;
  }
};
```

9.8 z

```
vector <int> Z;
void calculate_Z(string &s) {
  Z.clear();
  Z.resize(s.size(), 0);
  int l = 0, r = 0;
  for (int i = 1; i < s.size(); i++) {
    if (i <= r)
      Z[i] = min(Z[i - l], r - i + 1);
    while (i + Z[i] < s.size() && s[i + Z[i]] == s[Z[i]])
      Z[i]++;
    if (i + Z[i] - 1 > r) {
      l = i; r = i + Z[i] - 1;
    }
  }
}
void Z_matching(string &a, string &b) {
  string res = "";
  int i = 0;
  for (char ch : b)
    res[i++] = ch;
  res[i++] = '_';
  for (char ch : a)
    res[i++] = ch;
  calculate_Z(res);
  for (int i = 0; i < Z.size(); i++) {
    if (Z[i] == b.size())
      cout << "found a matching start at " << i - b.size() - 1
           << "\n";
  }
}
```

10 tree

10.1 Heavy Light Decomposition

```
const int N = 100005;
vector<int> G[N];
struct HLD {
  vector<int> pa, sz, depth, mxson, topf, id;
  int n, idcnt = 0;
  HLD(int _n) : n(_n), pa(_n + 1), sz(_n + 1), depth(_n + 1),
       mxson(_n + 1), topf(_n + 1), id(_n + 1) {}
  void dfs1(int v = 1, int p = -1) {
    pa[v] = p; sz[v] = 1; mxson[v] = 0;
    depth[v] = (p == -1 ? 0 : depth[p] + 1);
    for (int u : G[v]) {
      if (u == p) continue;
      dfs1(u, v);
      sz[v] += sz[u];
      if (sz[u] > sz[mxson[v]]) mxson[v] = u;
    }
  }
  void dfs2(int v = 1, int top = 1) {
    id[v] = ++idcnt;
    topf[v] = top;
    if (mxson[v]) dfs2(mxson[v], top);
    for (int u : G[v]) {
      if (u == mxson[v] || u == pa[v]) continue;
      dfs2(u, u);
    }
  }
  int path_query(int a, int b) {
    int res = 0;
    while (topf[a] != topf[b]) {
      if (depth[topf[a]] < depth[topf[b]]) swap(a, b);
      res = max(res, query(id[topf[a]], id[a])); // query : l =
            id[topf[a]], r = id[a]
      a = pa[topf[a]];
    }
    if (depth[a] < depth[b]) swap(a, b);
    res = max(res, query(id[b], id[a])); // query : l = id[b],
         r = id[a]
    return res;
  }
};
```